Article

Champion-level drone racing using deep reinforcement learning

https://doi.org/10.1038/s41586-023-06419-4

Received: 5 January 2023

Accepted: 10 July 2023

Published online: 30 August 2023

Open access

Check for updates

Elia Kaufmann[1]✉, Leonard Bauersfeld[1], Antonio Loquercio[1], Matthias Müller[2], Vladlen Koltun[3] & Davide Scaramuzza[1]

First-person view (FPV) drone racing is a televised sport in which professional competitors pilot high-speed aircraft through a 3D circuit. Each pilot sees the environment from the perspective of their drone by means of video streamed from an onboard camera. Reaching the level of professional pilots with an autonomous drone is challenging because the robot needs to fly at its physical limits while estimating its speed and location in the circuit exclusively from onboard sensors[1]. Here we introduce Swift, an autonomous system that can race physical vehicles at the level of the human world champions. The system combines deep reinforcement learning (RL) in simulation with data collected in the physical world. Swift competed against three human champions, including the world champions of two international leagues, in real-world head-to-head races. Swift won several races against each of the human champions and demonstrated the fastest recorded race time. This work represents a milestone for mobile robotics and machine intelligence[2], which may inspire the deployment of hybrid learning-based solutions in other physical systems.

Deep RL[3] has enabled some recent advances in artificial intelligence. Policies trained with deep RL have outperformed humans in complex competitive games, including Atari[4–6], Go[5,7–9], chess[5,9], StarCraft[10], Dota 2 (ref. 11) and Gran Turismo[12,13]. These impressive demonstrations of the capabilities of machine intelligence have primarily been limited to simulation and board-game environments, which support policy search in an exact replica of the testing conditions. Overcoming this limitation and demonstrating champion-level performance in physical competitions is a long-standing problem in autonomous mobile robotics and artificial intelligence[14–16].

FPV drone racing is a televised sport in which highly trained human pilots push aerial vehicles to their physical limits in high-speed agile manoeuvres (Fig. 1a). The vehicles used in FPV racing are quadcopters, which are among the most agile machines ever built (Fig. 1b). During a race, the vehicles exert forces that surpass their own weight by a factor of five or more, reaching speeds of more than 100 km h^{-1} and accelerations several times that of gravity, even in confined spaces. Each vehicle is remotely controlled by a human pilot who wears a headset showing a video stream from an onboard camera, creating an immersive 'first-person-view' experience (Fig. 1c).

Attempts to create autonomous systems that reach the performance of human pilots date back to the first autonomous drone racing competition in 2016 (ref. 17). A series of innovations followed, including the use of deep networks to identify the next gate location[18–20], transfer of racing policies from simulation to reality[21,22] and accounting for uncertainty in perception[23,24]. The 2019 AlphaPilot autonomous drone racing competition showcased some of the best research in the field[25]. However, the first two teams still took almost twice as long as a professional human pilot to complete the track[26,27]. More recently, autonomous systems have begun to reach expert human performance[28–30]. However, these works rely on near-perfect state estimation provided by an external motion-capture system. This makes the comparison with human pilots unfair, as humans only have access to onboard observations from the drone.

In this article, we describe Swift, an autonomous system that can race a quadrotor at the level of human world champions using only onboard sensors and computation. Swift consists of two key modules: (1) a perception system that translates high-dimensional visual and inertial information into a low-dimensional representation and (2) a control policy that ingests the low-dimensional representation produced by the perception system and produces control commands.

The control policy is represented by a feedforward neural network and is trained in simulation using model-free on-policy deep RL[31]. To bridge discrepancies in sensing and dynamics between simulation and the physical world, we make use of non-parametric empirical noise models estimated from data collected on the physical system. These empirical noise models have proved to be instrumental for successful transfer of the control policy from simulation to reality.

We evaluate Swift on a physical track designed by a professional drone-racing pilot (Fig. 1a). The track comprises seven square gates arranged in a volume of 30 × 30 × 8 m, forming a lap of 75 m in length. Swift raced this track against three human champions: Alex Vanover, the 2019 Drone Racing League world champion, Thomas Bitmatta, two-time MultiGP International Open World Cup champion, and Marvin Schaepper, three-time Swiss national champion. The quadrotors used by Swift and by the human pilots have the same weight, shape and propulsion. They are similar to drones used in international competitions.

The human pilots were given one week of practice on the race track. After this week of practice, each pilot competed against Swift in several head-to-head races (Fig. 1a,b). In each head-to-head race, two drones

[1]Robotics and Perception Group, University of Zurich, Zürich, Switzerland. [2]Intel Labs, Munich, Germany. [3]Intel Labs, Jackson, WY, USA. ✉e-mail: ekaufmann@ifi.uzh.ch

文章

使用深度强化学习的冠军级无人机竞速

https://doi.org/10.1038/s41586-023-06419-4

收稿日期：2023年1月5日

接受日期：2023年7月10日

在线发布日期：2023年8月30日

开放获取

检查更新

Elia Kaufmann[1]✉、Leonard Bauersfeld[1], Antonio Loquercio[1], Matthias Müller[2], Vladlen Koltun[3]与Davide Scaramuzza[1]

第一人称视角（FPV）无人机竞速是一项电视转播的体育运动，职业选手操控高速飞行器穿越三维赛道。每位飞行员通过机载摄像头传输的视频流，从自己无人机的视角观察环境。要让自主无人机达到职业飞行员的水平极具挑战性，因为机器人需要在物理极限下飞行，同时仅依靠机载传感器估计自身在赛道中的速度和位置[1]。在此，我们介绍Swift，一种能够以人类世界冠军水平驾驭实体飞行器竞速的自主系统。该系统将仿真中的深度强化学习（RL）与在物理世界中采集的数据相结合。Swift在真实世界的正面对决比赛中与三位人类冠军同场竞技，其中包括两个国际联赛的世界冠军。Swift在与每位人类冠军的多次比赛中均赢得数场，并展现出有记录以来的最快比赛时间。这项工作代表了移动机器人与机器智能的一个里程碑[2]，，或将启发基于混合学习的解决方案在其他物理系统中的部署。

深度强化学习[3]已经推动了人工智能领域的一些最新进展。通过深度强化学习训练的策略在复杂的竞争性游戏中已经超越了人类，包括Atari[4–6]、围棋[5, 7–9]、国际象棋[5,9]，星际争霸[10]，Dota 2（参考文献11）和Gran Turismo[12,13]。这些对机器智能能力的令人印象深刻的展示主要局限于仿真和棋盘游戏环境，这些环境支持在与测试条件完全相同的副本中进行策略搜索。克服这一局限性并在物理竞赛中展示冠军级性能，是自主移动机器人和人工智能领域一个长期存在的问题[14–16]。

FPV无人机竞速是一项电视转播的体育运动，训练有素的人类飞行员在高速敏捷机动中将飞行器推向物理极限（图1a）。FPV竞速中使用的飞行器是四旋翼飞行器，它们是有史以来建造的最敏捷的机器之一（图1b）。在比赛期间，这些飞行器施加的力超过其自身重量的五倍或更多，速度超过100 km h^{-1}，加速度达到重力的数倍，即使在受限空间中也是如此。每架飞行器由一名人类飞行员远程控制，该飞行员佩戴显示来自机载摄像头视频流的头戴式设备，从而营造出身临其境的“第一人称视角”体验（图1c）。

创建达到人类飞行员性能的自主系统的尝试可以追溯到2016年首届自主无人机竞速比赛（参考文献17）。随后出现了一系列创新，包括使用深度网络来识别下一个门的位置[18–20]、将竞速策略从仿真迁移到现实[21,22]以及考虑感知中的不确定性[23,24]。2019年AlphaPilot自主无人机竞速比赛展示了该领域一些最优秀的研究[25]。然而，前两支队伍完成赛道所需的时间仍几乎是专业人类飞行员的两倍[26,27]。最近，自主系统已开始达到专家级人类性能[28–30]。然而，这些工作依赖于近乎完美的状态估计，该估计由外部动作捕捉系统提供。这使得与人类飞行员的比较不公平，因为人类只能获取无人机上的机载观测。

在本文中，我们介绍了Swift，这是一个自主系统，仅使用机载传感器和计算即可让四旋翼以人类世界冠军的水平进行竞速。Swift由两个关键模块组成：(1)一个感知系统，将高维视觉和惯性信息转换为低维表示；(2)一个控制策略，接收感知系统产生的低维表示并生成控制指令。

该控制策略由前馈神经网络表示，并在仿真中使用无模型同策略深度强化学习进行训练[31]。为了弥合仿真与物理世界之间在感知和动力学方面的差异，我们利用了从物理系统上采集的数据估计得到的非参数经验噪声模型。这些经验噪声模型已被证明对于将控制策略从仿真成功迁移到现实至关重要。

我们在一条由专业无人机竞速飞行员设计的实体赛道上评估了Swift（图1a）。该赛道包含七扇方形门，分布在30 × 30 × 8 m的体积内，构成一圈长度为75 m的赛道。Swift在这条赛道上与三位人类冠军进行了竞速：Alex Vanover，2019年无人机竞速联盟世界冠军；Thomas Bitmatta，两届MultiGP国际公开世界杯冠军；以及Marvin Schaepper，三届瑞士全国冠军。Swift和人类飞行员使用的四旋翼具有相同的重量、形状和推进系统。它们类似于国际比赛中使用的无人机。

人类飞行员在赛道上进行了一周的练习。经过这一周的练习，每位飞行员都与Swift进行了数场正面对决比赛（图1a,b）。在每场正面对决比赛中，两架无人机

[1]苏黎世大学机器人与感知组，瑞士苏黎世。[2]英特尔实验室，德国慕尼黑。[3]英特尔实验室，Jackson，怀俄明州，美国。✉电子邮箱：ekaufmann@ifi.uzh.ch



Fig. 1 | Drone racing. a, Swift (blue) races head-to-head against Alex Vanover, the 2019 Drone Racing League world champion (red). The track comprises seven square gates that must be passed in order in each lap. To win a race, a competitor has to complete three consecutive laps before its opponent. **b**, A close-up view of Swift, illuminated with blue LEDs, and a human-piloted drone, illuminated with red LEDs. The autonomous drones used in this work rely only on onboard sensory measurements, with no support from external infrastructure, such as motion-capture systems. **c**, From left to right: Thomas Bitmatta, Marvin Schaepper and Alex Vanover racing their drones through the track. Each pilot wears a headset that shows a video stream transmitted in real time from a camera aboard their aircraft. The headsets provide an immersive 'first-person-view' experience. **c**, Photo by Regina Sablotny.

(one controlled by a human pilot and one controlled by Swift) start from a podium. The race is set off by an acoustic signal. The first vehicle that completes three full laps through the track, passing all gates in the correct order in each lap, wins the race.

Swift won several races against each of the human pilots and achieved the fastest race time recorded during the events. Our work marks the first time, to our knowledge, that an autonomous mobile robot achieved world-champion-level performance in a real-world competitive sport.

The Swift system

Swift uses a combination of learning-based and traditional algorithms to map onboard sensory readings to control commands. This mapping comprises two parts: (1) an observation policy, which distils high-dimensional visual and inertial information into a task-specific low-dimensional encoding, and (2) a control policy that transforms the encoding into commands for the drone. A schematic overview of the system is shown in Fig. 2.

The observation policy consists of a visual–inertial estimator[32,33] that operates together with a gate detector[26], which is a convolutional neural network that detects the racing gates in the onboard images. Detected gates are then used to estimate the global position and orientation of the drone along the race track. This is done by a camera-resectioning algorithm[34] in combination with a map of the track. The estimate of the global pose obtained from the gate detector is then combined with the estimate from the visual–inertial estimator by means of a Kalman filter, resulting in a more accurate representation of the robot's state. The control policy, represented by a two-layer perceptron, maps the output of the Kalman filter to control commands for the aircraft. The policy is trained using on-policy model-free deep RL[31] in simulation. During training, the policy maximizes a reward that combines progress towards the next racing gate[35] with a perception objective that rewards keeping the next gate in the field of view of the camera. Seeing the next gate is rewarded because it increases the accuracy of pose estimation.

Optimizing a policy purely in simulation yields poor performance on physical hardware if the discrepancies between simulation and reality are not mitigated. The discrepancies are caused primarily by two factors: (1) the difference between simulated and real dynamics and (2) the noisy estimation of the robot's state by the observation policy when provided with real sensory data. We mitigate these discrepancies by collecting a small amount of data in the real world and using this data to increase the realism of the simulator.

Specifically, we record onboard sensory observations from the robot together with highly accurate pose estimates from a motion-capture system while the drone is racing through the track. During this data-collection phase, the robot is controlled by a policy trained in simulation that operates on the pose estimates provided by the motion-capture system. The recorded data allow to identify the characteristic failure modes of perception and dynamics observed through the race track. These intricacies of failing perception and unmodelled dynamics are dependent on the environment, platform, track and sensors. The perception and dynamics residuals are modelled using Gaussian processes[36] and *k*-nearest-neighbour regression, respectively. The motivation behind this choice is that we empirically



图 1 | 无人机竞速。**a**，Swift（蓝色）与Alex Vanover，2019年无人机竞速联盟世界冠军（红色），进行正面交锋。赛道包含七个方形门，每圈必须按顺序通过。要赢得比赛，竞争者必须比对手先完成连续三圈。**b**，Swift的近距离视图，用蓝色LED灯照亮，以及一架人类驾驶无人机，用红色LED灯照亮。本工作中使用的自主无人机依赖仅依赖机载传感器测量，无需外部基础设施（如动作捕捉系统）的支持。**c**，从左到右：Thomas Bitmatta、Marvin Schaepper 和 Alex Vanover 驾驶无人机穿越赛道。每位飞手佩戴头戴式设备，该设备实时显示从其飞行器上摄像头传输的视频流。头戴式设备提供沉浸式“第一人称视角”体验。**c**，摄影：Regina Sablotny。

（一架由人类飞行员控制，一架由Swift控制）从起飞台出发。比赛由声音信号启动。第一辆完成赛道三整圈、每圈按正确顺序通过所有门的飞行器赢得比赛。

Swift在与每位人类飞行员的多次比赛中均获胜，并创造了赛事期间记录的最快比赛时间。据我们所知，我们的工作首次实现了自主移动机器人在真实世界竞技体育中达到世界冠军级表现。

Swift系统

Swift结合了基于学习的方法和传统算法，将机载传感读数映射为控制指令。这一映射包含两部分：(1) 观测策略，将高维视觉与惯性信息提炼为任务特定的低维编码；(2) 控制策略，将编码转换为无人机的控制指令。系统的示意图概览如图**2**所示。

观测策略包含一个视觉惯性估计器[32,33]，它与门检测器协同运行[26]，，门检测器是一个卷积神经网络，用于在机载图像中检测竞速门。检测到的竞速门随后用于估计无人机沿赛道的全局位置和朝向。这是通过相机重定位算法[34]结合赛道地图来完成的。然后，通过卡尔曼滤波器将门检测器获得的全局位姿估计与视觉惯性估计器的估计相结合，从而得到更精确的表示机器人状态。控制策略由两层感知机表示，将卡尔曼滤波器的输出映射为飞行器的控制指令。该策略在仿真中使用同策略无模型深度强化学习进行训练[31]。训练期间，策略最大化一个奖励，该奖励结合了向下一个竞速门前进的进度[35]与一个感知目标，后者奖励将下一个门保持在摄像头视场中。看到下一个门之所以受到奖励，是因为这提高了位姿估计的准确性。

若未缓解仿真与现实之间的差异，纯粹在仿真中优化策略会导致在物理硬件上的性能不佳。这些差异主要由两个因素造成：(1) 仿真动力学与真实动力学之间的差异；(2) 观测策略在获得真实传感数据时对机器人状态的有噪估计。我们通过在真实世界中收集少量数据，并利用这些数据提高仿真器的逼真度，从而缓解这些差异。

具体而言，当无人机在赛道上竞速时，我们记录机器人的机载感官观测，以及来自动作捕捉系统的高精度位姿估计。在此数据采集阶段，机器人由在仿真中训练的、基于动作捕捉系统提供的位姿估计运行的策略控制。所记录的数据能够识别出赛道上观测到的感知与动力学的特征性失效模式。这些感知失效和未建模动力学的复杂性取决于环境、平台、赛道和传感器。感知残差和动力学残差分别使用高斯过程[36]和*k*近邻回归进行建模。这一选择的动机在于我们根据经验

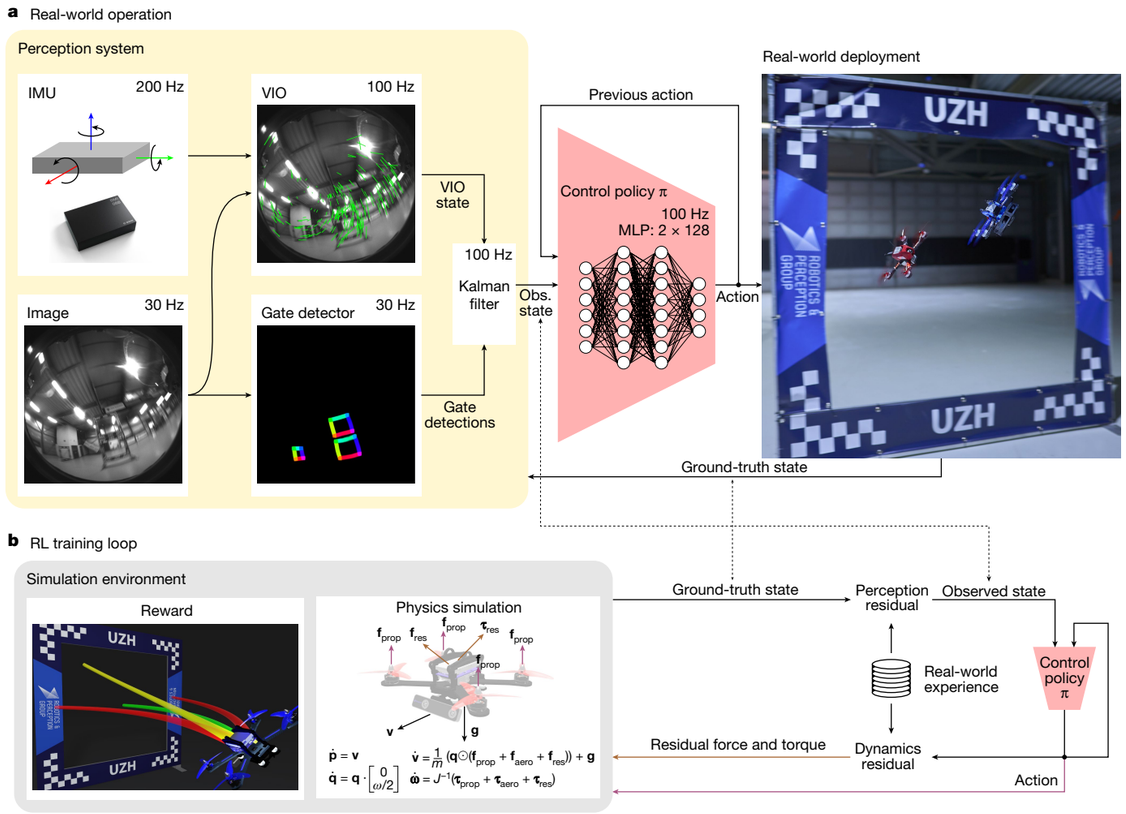


Fig. 2 | The Swift system. Swift consists of two key modules: a perception system that translates visual and inertial information into a low-dimensional state observation and a control policy that maps this state observation to control commands. Control commands specify desired collective thrust and body rates, the same control modality that the human pilots use. **a**, The perception system consists of a VIO module that computes a metric estimate of the drone state from camera images and high-frequency measurements obtained by an inertial measurement unit (IMU). The VIO estimate is coupled with a neural network that detects the corners of racing gates in the image stream. The corner detections are mapped to a 3D pose and fused with the VIO estimate using a Kalman filter. **b**, We use model-free on-policy deep RL to train the control policy in simulation. During training, the policy maximizes a reward that combines progress towards the centre of the next racing gate with a perception objective to keep the next gate in the field of view of the camera. To transfer the racing policy from simulation to the physical world, we augment the simulation with data-driven residual models of the vehicle's perception and dynamics. These residual models are identified from real-world experience collected on the race track. MLP, multilayer perceptron.

found perception residuals to be stochastic and dynamics residuals to be largely deterministic (Extended Data Fig. 1). These residual models are integrated into the simulation and the racing policy is fine-tuned in this augmented simulation. This approach is related to the empirical actuator models used for simulation-to-reality transfer in ref. 37 but further incorporates empirical modelling of the perception system and also accounts for the stochasticity in the estimate of the platform state.

We ablate each component of Swift in controlled experiments reported in the extended data. Also, we compare against recent work that tackles the task of autonomous drone racing with traditional methods, including trajectory planning and model predictive control (MPC). Although such approaches achieve comparable or even superior performance to our approach in idealized conditions, such as simplified dynamics and perfect knowledge of the robot's state, their performance collapses when their assumptions are violated. We find that approaches that rely on precomputed paths[28,29] are particularly sensitive to noisy perception and dynamics. No traditional method has achieved competitive lap times compared with Swift or human world champions, even when provided with highly accurate state estimation from a motion-capture system. Detailed analysis is provided in the extended data.

Results

The drone races take place on a track designed by an external world-class FPV pilot. The track features characteristic and challenging manoeuvres, such as a Split-S (Figs. 1a (top-right corner) and 4d). Pilots are allowed to continue racing even after a crash, provided their vehicle is still able to fly. If both drones crash and cannot complete the track, the drone that proceeded farther along the track wins.

As shown in Fig. 3b, Swift wins 5 out of 9 races against A. Vanover, 4 out of 7 races against T. Bitmatta and 6 out of 9 races against M. Schaepper. Out of the 10 losses recorded for Swift, 40% were because of a collision with the opponent, 40% because of collision with a gate and 20% because of the drone being slower than the human pilot. Overall, Swift

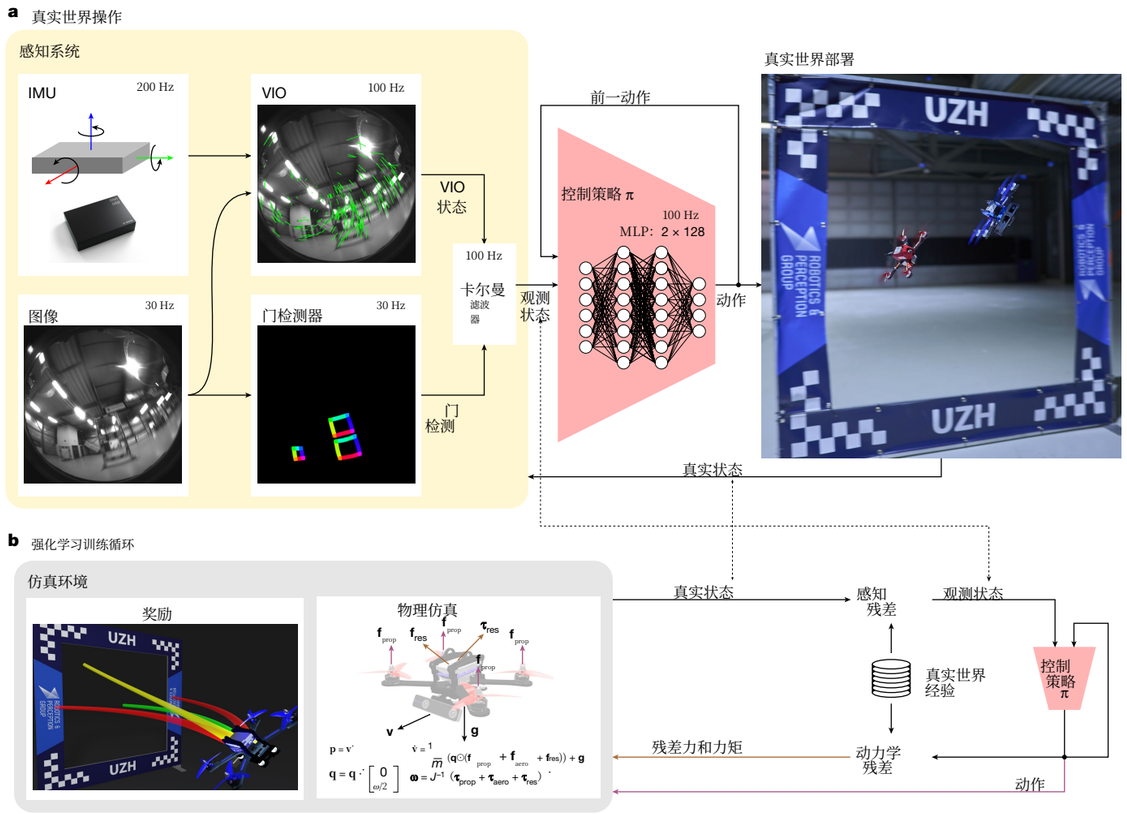


图 2 | Swift系统。 Swift由两个关键模块组成：一个感知系统，将视觉与惯性信息转换为低维状态观测；以及一个控制策略，将该状态观测映射为控制指令。控制指令指定期望的集体推力和机体角速率，与人类飞行员使用的控制模态相同。**a**，感知系统由一个VIO模块组成，该模块根据相机图像和惯性测量单元（IMU）获得的高频测量值计算无人机状态的度量估计。VIO估计与一个神经网络相结合，该神经网络在图像中检测竞速门的角点流。角点检测被映射为三维位姿，并使用卡尔曼滤波器与VIO估计融合。**b**，我们使用无模型同策略深度强化学习在仿真中训练控制策略。训练期间，策略最大化一个奖励，该奖励将朝向下一个赛道门中心的进展与保持下一个门位于摄像头视场内的感知目标相结合。为了将竞速策略从仿真迁移到物理世界，我们用车辆感知和动力学的数据驱动残差模型增强仿真。这些残差模型是从赛道上收集的真实世界经验中辨识得到的。MLP，多层感知机。

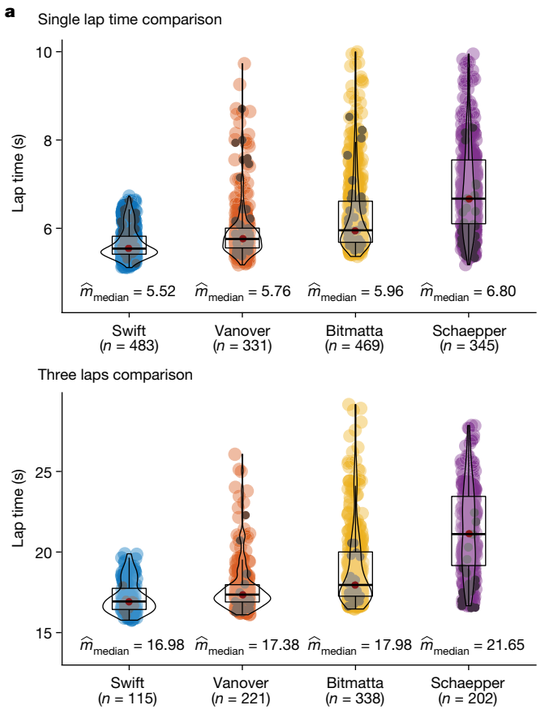
发现感知残差具有随机性，而动力学残差在很大程度上是确定性的（扩展数据图1）。这些残差模型被集成到仿真中，竞速策略在此增强仿真中进行微调。该方法与参考文献37中用于仿真到现实迁移的经验执行器模型相关，但进一步纳入了感知系统的经验建模，并且还考虑了平台状态估计中的随机性。

我们在扩展数据中报告的受控实验中对Swift的每个组件进行了消融。此外，我们与最近采用传统方法解决自主无人机竞速任务的工作进行了比较，包括轨迹规划和模型预测控制（MPC）。尽管此类方法在理想条件下（如简化动力学和完美已知机器人状态）能达到与我们的方法相当甚至更优的性能，但当其假设被违反时，性能便会崩溃。我们发现依赖预计算路径的方法[28,29]对噪声感知和动力学特别敏感。没有任何传统方法即使提供了来自动作捕捉系统的高精度状态估计，也未能达到与Swift或人类世界冠军相当的竞技圈速。详细分析见扩展数据。

结果

无人机竞速在由外部世界级FPV飞手设计的赛道上进行。赛道具有典型且富有挑战性的机动动作，例如Split-S（图1a（右上角）和图4d）。即使发生坠毁，只要飞行器仍能飞行，飞行员就可继续比赛。如果两架无人机均坠毁且无法完成赛道，则沿赛道行进更远的那架无人机获胜。

如图3b所示，Swift在对阵A. Vanover的9场比赛中赢得5场，在对阵T. Bitmatta的7场比赛中赢得4场，在对阵M. Schaepper的9场比赛中赢得6场。在Swift记录的10场负场中，40%是由于与对手碰撞，40%是由于与门碰撞，20%是由于无人机慢于人类飞行员。总体而言，Swift



b Head-to-head racing results

	Number of races	Best time-to-finish	Wins	Losses	Win ratio
A. Vanover versus Swift	9	17.956 s	4	5	0.44
T. Bitmatta versus Swift	7	18.746 s	3	4	0.43
M. Schaepper versus Swift	9	21.160 s	3	6	0.33
Swift versus human pilots	25	17.465 s	15	10	0.60

Fig. 3 | Results. a, Lap-time results. We compare Swift against the human pilots in time-trial races. Lap times indicate best single lap times and best average times achieved in a heat of three consecutive laps. The reported statistics are computed over a dataset recorded during one week on the race track, which corresponds to 483 (115) data points for Swift, 331 (221) for A. Vanover, 469 (338) for T. Bitmatta and 345 (202) for M. Schaepper. The first number is the number of single laps and the second is the number of three consecutive laps. The dark points in each distribution correspond to laps flown in race conditions. **b**, Head-to-head results. We report the number of head-to-head races flown by each pilot, the number of wins and losses, as well as the win ratio.

wins most races against each human pilot. Swift also achieves the fastest race time recorded, with a lead of half a second over the best time clocked by a human pilot (A. Vanover).

Figure 4 and Extended Data Table 1d provide an analysis of the fastest lap flown by Swift and each human pilot. Although Swift is globally faster than all human pilots, it is not faster on all individual segments of the track (Extended Data Table 1). Swift is consistently faster at the start and in tight turns such as the split S. At the start, Swift has a lower reaction time, taking off from the podium, on average, 120 ms before human pilots. Also, it accelerates faster and reaches higher speeds going into the first gate (Extended Data Table 1d, segment 1). In sharp turns, as shown in Fig. 4c,d, Swift finds tighter manoeuvres. One hypothesis is that Swift optimizes trajectories on a longer timescale than human pilots. It is known that model-free RL can optimize long-term rewards through a value function[38]. Conversely, human pilots plan their motion on a shorter timescale, up to one gate into the future[39]. This is apparent, for example in the split S (Fig. 4b,d), for which human pilots are faster in the beginning and at the end of the manoeuvre, but slower overall (Extended Data Table 1d, segment 3). Also, human pilots orient the aircraft to face the next gate earlier than Swift does (Fig. 4c,d). We propose that human pilots are accustomed to keeping the upcoming gate in view, whereas Swift has learned to execute some manoeuvres while relying on other cues, such as inertial data and visual odometry against features in the surrounding environments. Overall, averaged over the entire track, the autonomous drone achieves the highest average speed, finds the shortest racing line and manages to maintain the aircraft closer to its actuation limits throughout the race, as indicated by the average thrust and power drawn (Extended Data Table 1d).

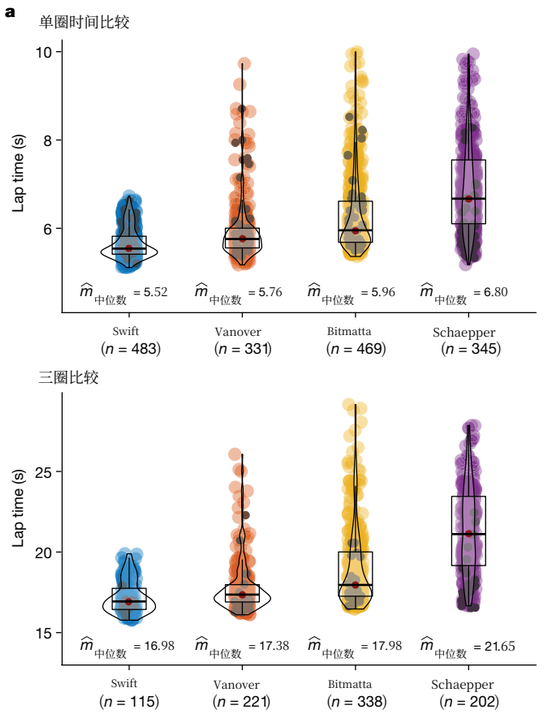
We also compare the performance of Swift and the human champions in time trials (Fig. 3a). In a time trial, a single pilot races the track, with the number of laps left to the discretion of the pilot. We accumulate time-trial data from the practice week and the races, including training runs (Fig. 3a, coloured) and laps flown in race conditions (Fig. 3a, black). For each contestant, we use more than 300 laps for computing statistics. The autonomous drone more consistently pushes for fast lap times, exhibiting lower mean and variance. Conversely, human pilots decide whether to push for speed on a lap-by-lap basis, yielding higher mean and variance in lap times, both during training and in the races. The ability to adapt the flight strategy allows human pilots to maintain a slower pace if they identify that they have a clear lead, so as to reduce the risk of a crash. The autonomous drone is unaware of its opponent and pushes for fastest expected completion time no matter what, potentially risking too much when in the lead and too little when trailing behind[40].

Discussion

FPV drone racing requires real-time decision-making based on noisy and incomplete sensory input from the physical environment. We have presented an autonomous physical system that achieves champion-level performance in this sport, reaching—and at times exceeding—the performance of human world champions. Our system has certain structural advantages over the human pilots. First, it makes use of inertial data from an onboard inertial measurement unit[32]. This is akin to the human vestibular system[41], which is not used by the human pilots because they are not physically in the aircraft and do not feel the accelerations acting on it. Second, our system benefits from lower sensorimotor latency (40 ms for Swift versus an average of 220 ms for expert human pilots[39]). On the other hand, the limited refresh rate of the camera used by Swift (30 Hz) can be considered a structural advantage for human pilots, whose cameras' refresh rate is four times as fast (120 Hz), improving their reaction time[42].

Human pilots are impressively robust: they can crash at full speed, and—if the hardware still functions—carry on flying and complete the track. Swift was not trained to recover after a crash. Human pilots are also robust to changes in environmental conditions, such as illumination, which can markedly alter the appearance of the track. By contrast, Swift's perception system assumes that the appearance of the environment is consistent with what was observed during training. If this assumption fails, the system can fail. Robustness to appearance changes can be provided by training the gate detector and the residual observation model in a diverse set of conditions. Addressing these limitations could enable applying the presented approach in autonomous drone racing competitions in which access to the environment and the drone is limited[25].

Notwithstanding the remaining limitations and the work ahead, the attainment by an autonomous mobile robot of world-champion-level performance in a popular physical sport is a milestone for robotics



b 正面对决竞速结果

	数量 比赛次数	最佳 完赛时间	胜场	负场	Win 比率
A. Vanover 对阵 Swift	9	17.956 s	4	5	0.44
T. Bitmatta 对阵 Swift	7	18.746 s	3	4	0.43
M. Schaepper 对阵 Swift	9	21.160 s	3	6	0.33
Swift 对比人类飞行员	25	17.465 s	15	10	0.60

图 3 | 结果。a，单圈时间结果。我们将Swift与人类飞行员在计时赛中进行比较。单圈时间表示在连续三圈的轮次中取得的最佳单圈最佳时间和最佳平均时间。所报告的统计数据是在赛道上为期一周记录的数据集上计算得出的，对应Swift的483（115）个数据点，A. Vanover的331（221）个，T. Bitmatta的469（338）个，以及M. Schaepper的345（202）个。第一个数字是单圈数，第二个是连续三圈数。每个分布中的深色点对应于在比赛条件下飞行的圈。**b**，正面对决结果。我们报告每位飞行员飞行的正面对决比赛数量、胜场数和负场数，以及胜率。

在对阵每位人类飞行员的比赛中赢得最多比赛。Swift还实现了记录的最快比赛时间，比人类飞行员（A. Vanover）创造的最佳时间领先半秒。

图4和扩展数据表1d分析了Swift和每位人类飞行员飞出的最快圈速。尽管Swift在全局上快于所有人类飞行员，但在赛道的各个单独赛段上并非都更快（扩展数据表1）。Swift在起步阶段以及诸如半滚倒转等急转弯中始终更快。起步时，Swift的反应时间更短，从出发台上起飞平均比人类飞行员早120 ms。此外，它加速更快，并在进入第一个门时达到更高的速度（扩展数据表1d，赛段1）。在急转弯中，如图4c,d所示，Swift找到了更紧凑的机动动作。一种假设是，Swift在比人类飞行员更长的时间尺度上优化轨迹。已知无模型强化学习可以通过价值函数优化

长期奖励[38]。相反，人类飞行员在更短的时间尺度上规划其运动，最多提前一个门[39]。这一点很明显，例如在半滚倒转中（图4b,d），人类飞行员在机动动作开始时和结束时更快，但总体更慢（扩展数据表1d，赛段3）。此外，人类飞行员比Swift更早调整飞行器朝向以面向下一个门（图4c,d）。我们提出，人类飞行员习惯于将即将到来的门保持在视野中，而Swift已学会在依赖其他线索执行某些机动动作，例如惯性数据以及针对周围环境特征的视觉里程计。总体而言，在整个赛道上取平均，自主无人机实现了最高的平均速度，找到了最短的赛车线，并在整个比赛过程中设法将飞行器保持在更接近其驱动极限的状态，如平均推力和功率消耗所示（扩展数据表1d）。

我们还比较了 Swift 与人类冠军在计时赛中的表现（图 3a）。在计时赛中，由一名飞行员单独跑赛道，剩余圈数由飞行员自行决定。我们汇总了练习周和比赛期间的计时赛数据，包括训练飞行（图 3a，彩色）以及比赛条件下飞行的圈（图 3a，黑色）。对于每位参赛者，我们使用超过 300 圈来计算统计数据。自主无人机更一致地追求快速圈速，表现出更低的均值和方差。相反，人类飞行员则逐圈决定是否加速，因此在训练和比赛中圈速都具有更高的均值和方差。调整飞行策略的能力使人类飞行员在判断自己已明显领先时能够保持较慢节奏，以降低坠毁风险。自主无人机不知道对手的情况，无论何时都追求最快预期完成时间，这可能在其领先时冒太大风险，而在落后时又冒太小风险。[40]。

讨论

FPV无人机竞速要求基于来自物理环境的噪声和不完整的感官输入进行实时决策。我们提出了一种自主物理系统，在该运动中达到了冠军级表现，达到——有时甚至超过——人类世界冠军的表现。我们的系统相较于人类飞行员具有某些结构性优势。首先，它利用了来自机载惯性测量单元的惯性数据[32]。这类似于人类前庭系统[41]，，而人类飞行员并不使用该系统，因为他们并非实际位于飞行器内，无法感受到作用于其上的加速度。其次，我们的系统受益于更低的感觉运动延迟（Swift为40 ms，而专家级人类飞行员平均为220 ms[39]）。另一方面，Swift所用摄像头的有限刷新率（30 Hz）可被视为人类飞行员的结构性优势，其摄像头的刷新率快四倍（120 Hz），从而改善了他们的反应时间[42]。

人类飞行员具有令人印象深刻的鲁棒性：他们能够以全速坠毁，并且——如果硬件仍然正常工作——继续飞行并完成赛道。Swift并未接受过坠毁后恢复的训练。人类飞行员对环境条件的变化（例如光照）也具有鲁棒性，而光照会显著改变赛道的外观。相比之下，Swift的感知系统假设环境的外观与训练期间观测到的一致。如果这一假设不成立，系统可能会失效。通过对门检测器和残差观测模型在多样化条件下进行训练，可以提供对外观变化的鲁棒性。解决这些局限性将使得所提出的方法能够应用于自主无人机竞速比赛中，而在这些比赛中，对环境和无人机的访问是受限的[25]。

尽管仍存在局限性和有待完成的工作，但自主移动机器人在一项流行的物理运动中达到世界冠军级表现，是机器人学的一个里程碑

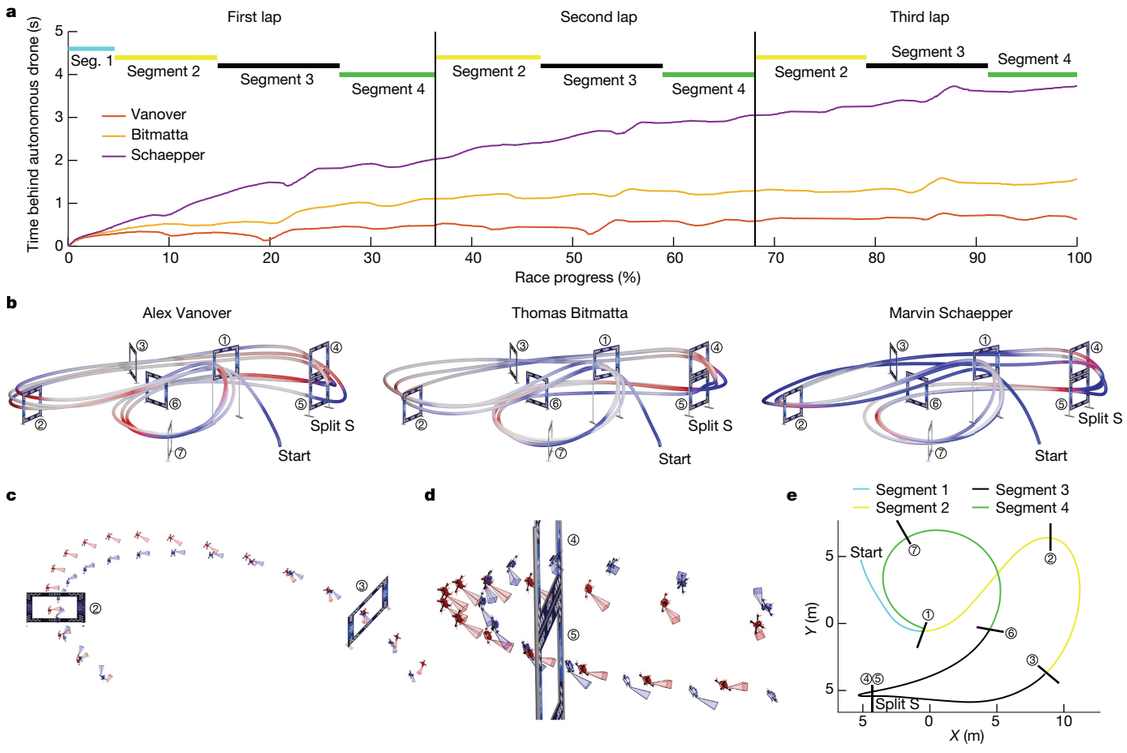


Fig. 4 | Analysis. a, Comparison of the fastest race of each pilot, illustrated by the time behind Swift. The time difference from the autonomous drone is computed as the time since it passed the same position on the track. Although Swift is globally faster than all human pilots, it is not necessarily faster on all individual segments of the track. **b**, Visualization of where the human pilots are faster (red) and slower (blue) compared with the autonomous drone. Swift is consistently faster at the start and in tight turns, such as the split S. **c**, Analysis of the manoeuvre after gate 2. Swift in blue, Vanover in red. Swift gains time against human pilots in this segment as it executes a tighter turn while maintaining comparable speed. **d**, Analysis of the split S manoeuvre. Swift in blue, Vanover in red. The split S is the most challenging segment in the race track, requiring a carefully coordinated roll and pitch motion that yields a descending half-loop through the two gates. Swift gains time against human pilots on this segment as it executes a tighter turn with less overshoot. **e**, Illustration of track segments used for analysis. Segment 1 is traversed once at the start, whereas segments 2–4 are traversed in each lap (three times over the course of a race).

and machine intelligence. This work may inspire the deployment of hybrid learning-based solutions in other physical systems, such as autonomous ground vehicles, aircraft and personal robots, across a broad range of applications.

Online content

Any methods, additional references, Nature Portfolio reporting summaries, source data, extended data, supplementary information, acknowledgements, peer review information; details of author contributions and competing interests; and statements of data and code availability are available at https://doi.org/10.1038/s41586-023-06419-4.

1. De Wagter, C., Paredes-Vallés, F., Sheth, N. & de Croon, G. Learning fast in autonomous drone racing. *Nat. Mach. Intell.* **3**, 923 (2021).
2. Hanover, D. et al. Autonomous drone racing: a survey. Preprint at https://arxiv.org/abs/2301.01755 (2023).
3. Sutton, R. S. & Barto, A. G. *Reinforcement Learning: An Introduction* (MIT Press, 2018).
4. Mnih, V. et al. Human-level control through deep reinforcement learning. *Nature* **518**, 529–533 (2015).
5. Schrittwieser, J. et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature* **588**, 604–609 (2020).
6. Ecoffet, A., Huizinga, J., Lehman, J., Stanley, K. O. & Clune, J. First return, then explore. *Nature* **590**, 580–586 (2021).
7. Silver, D. et al. Mastering the game of Go with deep neural networks and tree search. *Nature* **529**, 484–489 (2016).
8. Silver, D. et al. Mastering the game of Go without human knowledge. *Nature* **550**, 354–359 (2017).
9. Silver, D. et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science* **362**, 1140–1144 (2018).
10. Vinyals, O. et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature* **575**, 350–354 (2019).
11. Berner, C. et al. Dota 2 with large scale deep reinforcement learning. Preprint at https://arxiv.org/abs/1912.06680 (2019).
12. Fuchs, F., Song, Y., Kaufmann, E., Scaramuzza, D. & Dürr, P. Super-human performance in Gran Turismo Sport using deep reinforcement learning. *IEEE Robot. Autom. Lett.* **6**, 4257–4264 (2021).
13. Wurman, P. R. et al. Outracing champion Gran Turismo drivers with deep reinforcement learning. *Nature* **602**, 223–228 (2022).
14. Funke, J. et al. in *Proc. 2012 IEEE Intelligent Vehicles Symposium* 541–547 (IEEE, 2012).
15. Spielberg, N. A., Brown, M., Kapania, N. R., Kegelman, J. C. & Gerdes, J. C. Neural network vehicle models for high-performance automated driving. *Sci. Robot.* **4**, eaaw1975 (2019).
16. Won, D.-O., Müller, K.-R. & Lee, S.-W. An adaptive deep reinforcement learning framework enables curling robots with human-like performance in real-world conditions. *Sci. Robot.* **5**, eabb9764 (2020).
17. Moon, H., Sun, Y., Baltes, J. & Kim, S. J. The IROS 2016 competitions. *IEEE Robot. Autom. Mag.* **24**, 20–29 (2017).
18. Jung, S., Hwang, S., Shin, H. & Shim, D. H. Perception, guidance, and navigation for indoor autonomous drone racing using deep learning. *IEEE Robot. Autom. Lett.* **3**, 2539–2544 (2018).
19. Kaufmann, E. et al. in *Proc. 2nd Conference on Robot Learning (CoRL)* 133–145 (PMLR, 2018).
20. Zhang, D. & Doyle, D. D. in *Proc. 2020 IEEE Aerospace Conference*, 1–11 (IEEE, 2020).

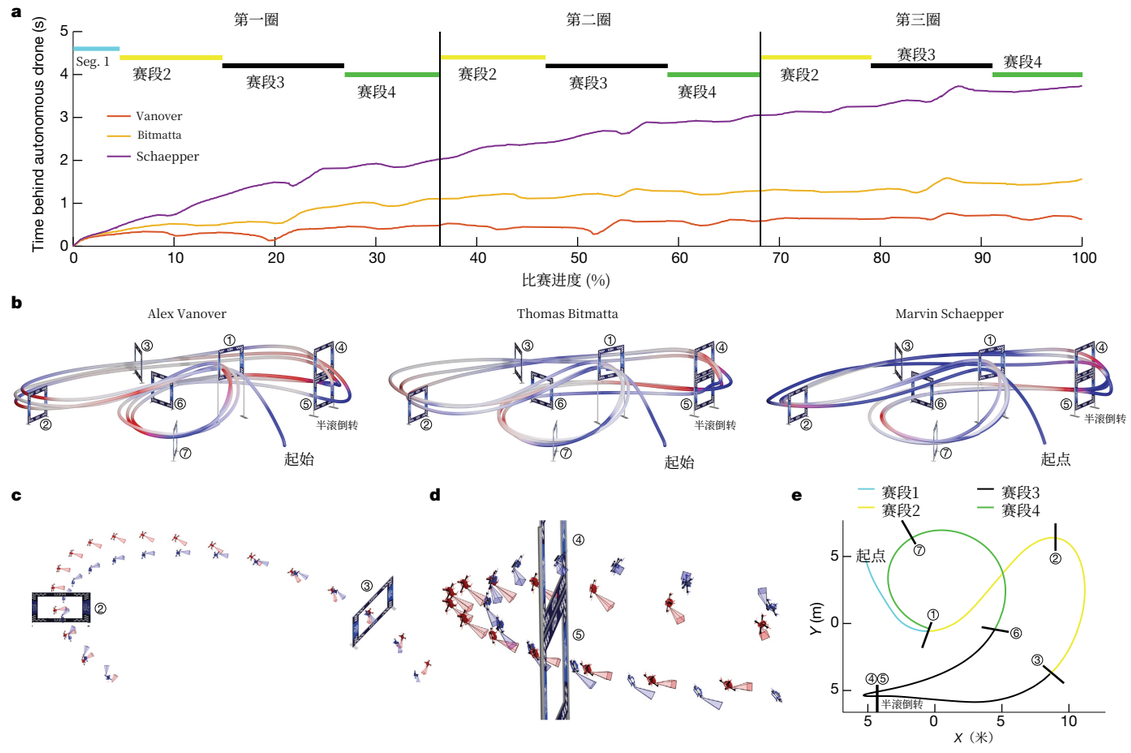


图 4 | 分析。**a**，各飞行员最快比赛对比，以落后 Swift 的时间表示。与自主无人机的时间差按其通过赛道上同一位置后的时间计算。尽管 Swift 全局上快于所有人类飞行员，但在赛道的各个单独赛段上未必都更快。**b**，与自主无人机相比，人类飞行员更快（红色）和更慢（蓝色）位置的可视化。Swift 在起步和急转弯（如半滚倒转）处始终更快。**c**，门2之后机动动作的分析。Swift 为蓝色，Vanover 为红色。Swift 在该赛段中通过执行更紧的转弯同时对人类飞行员取得时间优势，同时保持相当的速度。**d**，半滚倒转机动动作的分析。Swift 为蓝色，Vanover 为红色。半滚倒转是赛道中最具挑战性的赛段，需要精心协调的滚转和俯仰运动，从而通过两个门形成下降半滚。Swift 在该赛段中通过以更小的过冲执行更紧的转弯，对人类飞行员取得时间优势。**e**，用于分析的赛道赛段示意图。赛段1在起步时通过一次，而赛段2–4在每一圈中通过（比赛过程中共三次）。

与机器智能。这项工作可能会启发在其他物理系统中部署基于混合学习的解决方案，例如自动驾驶地面车辆、飞行器和个人机器人，应用于广泛领域。

在线内容

所有方法、附加参考文献、Nature Portfolio 报告摘要、源数据、扩展数据、补充信息、致谢、同行评审信息；作者贡献和利益冲突详情；以及数据和代码可用性声明均可在 https://doi.org/10.1038/s41586-023-06419-4 获取。

1. De Wagter, C., Paredes-Vallés, F., Sheth, N. & de Croon, G. 在自主无人机竞速中快速学习。*Nat. Mach. Intell.* **3**，923 (2021)。2. Hanover, D. 等。自主无人机竞速：综述。预印本见 https://arxiv.org/abs/2301.01755 (2023)。3. Sutton, R. S. & Barto, A. G. 强化学习：导论 (MIT Press, 2018)。4. Mnih, V. 等。通过深度强化学习实现人类水平的控制。*Nature* **518**，529–533 (2015)。5. Schrittwieser, J. 等。通过使用学习模型进行规划来掌握 Atari 、围棋、国际象棋和将棋。*Nature* **588**，604–609 (2020)。6. Ecoffet, A., Huizinga, J., Lehman, J., Stanley, K. O. & Clune, J. 先返回，再探索。*Nature* **590**，580–586 (2021)。7. Silver, D. 等。利用深度神经网络和树搜索掌握围棋游戏。Nature 529, 484–489 (2016). 8. Silver, D. 等。无需人类知识掌握围棋游戏。Nature 550, 354–359 (2017). 9. Silver, D. 等。一种通过自我对弈掌握国际象棋、将棋和围棋的通用强化学习算法。Science 362, 1140–1144 (2018). 10. Vinyals, O. 等。使用多智能体强化学习在 StarCraft II 中达到大师级水平。Nature 575, 350–354 (2019). 11. Berner, C. 等。大规模深度强化学习下的 Dota 2。预印本见 https://arxiv.org/abs/1912.06680 (2019). 12. Fuchs, F., Song, Y., Kaufmann, E., Scaramuzza, D. & Dürr, P. 使用深度强化学习在 Gran Turismo Sport 中实现超人表现。IEEE Robot. Autom. Lett. 6, 4257–4264 (2021). 13. Wurman, P. R. 等。使用深度强化学习超越冠军 Gran Turismo 车手。Nature 602, 223–228 (2022). 14. Funke, J. 等。载于 Proc. 2012 IEEE Intelligent Vehicles Symposium 541–547 (IEEE, 2012). 15. Spielberg, N. A., Brown, M., Kapania, N. R., Kegelman, J. C. & Gerdes, J. C. 用于高性能自动驾驶的神经网络车辆模型。Sci. Robot. 4, eaaw1975 (2019). 16. Won, D.-O., Müller, K.-R. & Lee, S.-W. 一种自适应深度强化学习框架使冰壶机器人在真实世界条件下具备类人表现。Sci. Robot. 5, eabb9764 (2020). 17. Moon, H., Sun, Y., Baltes, J. & Kim, S. J. IROS 2016 竞赛。IEEE Robot. Autom. Mag. 24, 20–29 (2017). 18. Jung, S., Hwang, S., Shin, H. & Shim, D. H. 使用深度学习进行室内自主无人机竞速的感知、制导和导航。IEEE Robot. Autom. Lett. 3, 2539–2544 (2018). 19. Kaufmann, E. 等。载于 Proc. 2nd Conference on Robot Learning (CoRL) 133–145 (PMLR, 2018). 20. Zhang, D. & Doyle, D. D. 载于 Proc. 2020 IEEE Aerospace Conference, 1–11 (IEEE, 2020).

21. Loquercio, A. et al. Deep drone racing: from simulation to reality with domain randomization. *IEEE Trans. Robot.* **36**, 1–14 (2019).
22. Loquercio, A. et al. Learning high-speed flight in the wild. *Sci. Robot.* **6**, eabg5810 (2021).
23. Kaufmann, E. et al. in *Proc. 2019 International Conference on Robotics and Automation (ICRA)* 690–696 (IEEE, 2019).
24. Li, S., van der Horst, E., Duernay, P., De Wagter, C. & de Croon, G. C. Visual model-predictive localization for computationally efficient autonomous racing of a 72-g drone. *J. Field Robot.* **37**, 667–692 (2020).
25. A.I. is flying drones (very, very slowly). https://www.nytimes.com/2019/03/26/technology/alphapilot-ai-drone-racing.html (2019).
26. Foehn, P. et al. AlphaPilot: autonomous drone racing. *Auton. Robots* **46**, 307–320 (2021).
27. Wagter, C. D., Paredes-Vallé, F., Sheth, N. & de Croon, G. The sensing, state-estimation, and control behind the winning entry to the 2019 Artificial Intelligence Robotic Racing Competition. *Field Robot.* **2**, 1263–1290 (2022).
28. Foehn, P., Romero, A. & Scaramuzza, D. Time-optimal planning for quadrotor waypoint flight. *Sci. Robot.* **6**, eabh1221 (2021).
29. Romero, A., Sun, S., Foehn, P. & Scaramuzza, D. Model predictive contouring control for time-optimal quadrotor flight. *IEEE Trans. Robot.* **38**, 3340–3356 (2022).
30. Sun, S., Romero, A., Foehn, P., Kaufmann, E. & Scaramuzza, D. A comparative study of nonlinear MPC and differential-flatness-based control for quadrotor agile flight. *IEEE Trans. Robot.* **38**, 3357–3373 (2021).
31. Schulman, J., Wolski, F., Dhariwal, P., Radford, A. & Klimov, O. Proximal policy optimization algorithms. Preprint at https://arxiv.org/abs/1707.06347 (2017).
32. Scaramuzza, D. & Zhang, Z. *Encyclopedia of Robotics* (eds Ang, M., Khatib, O. & Siciliano, B.) 1–9 (Springer, 2019).
33. Huang, G. in *Proc. 2019 International Conference on Robotics and Automation (ICRA)* 9572–9582 (IEEE, 2019).
34. Collins, T. & Bartoli, A. Infinitesimal plane-based pose estimation. *Int. J. Comput. Vis.* **109**, 252–286 (2014).
35. Song, Y., Steinweg, M., Kaufmann, E. & Scaramuzza, D. in *Proc. 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* 1205–1212 (IEEE, 2021).
36. Williams, C. K. & Rasmussen, C. E. *Gaussian Processes for Machine Learning* (MIT Press, 2006).
37. Hwangbo, J. et al. Learning agile and dynamic motor skills for legged robots. *Sci. Robot.* **4**, eaau5872 (2019).
38. Hung, C.-C. et al. Optimizing agent behavior over long time scales by transporting value. *Nat. Commun.* **10**, 5223 (2019).
39. Pfeiffer, C. & Scaramuzza, D. Human-piloted drone racing: visual processing and control. *IEEE Robot. Autom. Lett.* **6**, 3467–3474 (2021).
40. Spica, R., Cristofalo, E., Wang, Z., Montijano, E. & Schwager, M. A real-time game theoretic planner for autonomous two-player drone racing. *IEEE Trans. Robot.* **36**, 1389–1403 (2020).
41. Day, B. L. & Fitzpatrick, R. C. The vestibular system. *Curr. Biol.* **15**, R583–R586 (2005).
42. Kim, J. et al. Esports arms race: latency and refresh rate for competitive gaming tasks. *J. Vis.* **19**, 218c (2019).

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

© The Author(s) 2023

21. Loquercio, A. 等。深度无人机竞速：通过域随机化从仿真到现实。IEEE Trans. Robot. 36, 1–14 (2019)。 22. Loquercio, A. 等。在野外环境中学习高速飞行。Sci. Robot. 6, eabg5810 (2021)。 23. Kaufmann, E. 等。载于 Proc. 2019 International Conference on Robotics and Automation (ICRA) 690–696 (IEEE, 2019)。 24. Li, S., van der Horst, E., Duernay, P., De Wagter, C. & de Croon, G. C. 面向计算高效自主竞速的 72 克无人机视觉模型预测定位。J. Field Robot. 37, 667–692 (2020)。 25. A.I. 正在驾驶无人机（非常、非常缓慢地）。https://www.nytimes.com/2019/03/26/technology/ alphapilot-ai-drone-racing.html (2019)。 26. Foehn, P. 等。AlphaPilot：自主无人机竞速。Auton. Robots 46, 307–320 (2021)。 27. Wagter, C. D., Paredes-Vallé, F., Sheth, N. & de Croon, G. 2019 Artificial Intelligence Robotic Racing Competition 获胜方案背后的感知、状态估计与控制。Field Robot. 2, 1263–1290 (2022)。 28. Foehn, P., Romero, A. & Scaramuzza, D. 四旋翼航点飞行的时间最优规划。Sci. Robot. 6, eabh1221 (2021)。 29. Romero, A., Sun, S., Foehn, P. & Scaramuzza, D. 面向时间最优四旋翼飞行的模型预测轮廓控制。IEEE Trans. Robot. 38, 3340–3356 (2022)。 30. Sun, S., Romero, A., Foehn, P., Kaufmann, E. & Scaramuzza, D. 非线性 MPC 与基于微分平坦的控制在四旋翼敏捷飞行中的比较研究。IEEE Trans. Robot. 38, 3357–3373 (2021)。 31. Schulman, J., Wolski, F., Dhariwal, P., Radford, A. & Klimov, O. 近端策略优化算法。预印本见 https://arxiv.org/abs/1707.06347 (2017)。 32. Scaramuzza, D. & Zhang, Z. Encyclopedia of Robotics（编：Ang, M., Khatib, O. & Siciliano, B.）1–9 (Springer, 2019)。 33. Huang, G. 载于 Proc. 2019 International Conference on Robotics and Automation (ICRA) 9572–9582 (IEEE, 2019)。 34. Collins, T. & Bartoli, A. 基于无穷小平面的位姿估计。Int. J. Comput. Vis. 109, 252–286 (2014)。 35. Song, Y., Steinweg, M., Kaufmann, E. & Scaramuzza, D. 载于 Proc. 2021 IEEE/RSJ International Conference o n Intelligent Robots and Systems (IROS) 1205–1212 (IEEE, 2021)。
36. Williams, C. K. & Rasmussen, C. E. 《机器学习中的高斯过程》(MIT Press, 2006)。 37. Hwangbo, J. 等。为腿式机器人学习敏捷且动态的运动技能。《Sci. Robot.》**4**，eaau5872 (2019)。 38. Hung, C.-C. 等。通过传递价值在长时间尺度上优化智能体行为。《Nat. Commun.》**10**，5223 (2019)。 39. Pfeiffer, C. & Scaramuzza, D. 人类驾驶无人机竞速：视觉处理与控制。《IEEE Robot. Autom. Lett.》**6**，3467–3474 (2021)。 40. Spica, R., Cristofalo, E., Wang, Z., Montijano, E. & Schwager, M. 面向自主双人无人机竞速的实时博弈论规划器。《IEEE Trans. Robot.》**36**，1389–1403 (2020)。 41. Day, B. L. & Fitzpatrick, R. C. 前庭系统。《Curr. Biol.》**15**，R583–R586 (2005)。 42. Kim, J. 等。电子竞技军备竞赛：竞技游戏任务中的延迟与刷新率。《J. Vis.》**19**，218c (2019)。

出版者注 施普林格·自然对已出版地图中的管辖权主张和机构隶属关系保持中立。

开放获取 本文根据知识共享署名4.0国际许可协议授权，允许以任何媒介或格式使用、共享、改编、分发和复制，只要您给予适当

注明原始作者和来源，提供知识共享许可协议的链接，并说明是否进行了修改。本文中的图像或其他第三方材料均包含在文章的知识共享许可协议中，除非在材料的署名行中另有说明。如果材料未包含在文章的知识共享许可协议中，且您的预期使用未获法定许可或超出许可范围，您需要直接向版权持有人获得许可。要查看此许可协议的副本，请访问 http://creativecommons.org/licenses/by/4.0/。

© 作者 2023

Article

Methods

Quadrotor simulation

Quadrotor dynamics. To enable large-scale training, we use a high-fidelity simulation of the quadrotor dynamics. This section briefly explains the simulation. The dynamics of the vehicle can be written as

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{\mathbf{p}}_{\mathcal{WB}} \\ \dot{\mathbf{q}}_{\mathcal{WB}} \\ \dot{\mathbf{v}}_{\mathcal{W}} \\ \dot{\boldsymbol{\omega}}_{\mathcal{B}} \\ \dot{\Omega} \end{bmatrix} = \begin{bmatrix} \mathbf{v}_{\mathcal{W}} \\ \mathbf{q}_{\mathcal{WB}} \cdot \begin{bmatrix} 0 \\ \boldsymbol{\omega}_{\mathcal{B}}/2 \end{bmatrix} \\ \frac{1}{m}(\mathbf{q}_{\mathcal{WB}} \odot (\mathbf{f}_{\text{prop}} + \mathbf{f}_{\text{aero}})) + \mathbf{g}_{\mathcal{W}} \\ J^{-1}(\boldsymbol{\tau}_{\text{prop}} + \boldsymbol{\tau}_{\text{mot}} + \boldsymbol{\tau}_{\text{aero}} + \boldsymbol{\tau}_{\text{iner}}) \\ \frac{1}{k_{\text{mot}}}(\Omega_{\text{ss}} - \Omega) \end{bmatrix}, \tag{1}$$

in which $\odot$ represents quaternion rotation, $\mathbf{p}_{\mathcal{WB}}$, $\mathbf{q}_{\mathcal{WB}}$, $\mathbf{v}_{\mathcal{W}}$ and $\boldsymbol{\omega}_{\mathcal{B}}$ denote the position, attitude quaternion, inertial velocity and body rates of the quadcopter, respectively. The motor time constant is k_{mot} and the motor speeds $\boldsymbol{\Omega}$ and $\boldsymbol{\Omega}_{\text{ss}}$ are the actual and steady-state motor speeds, respectively. The matrix J is the inertia of the quadcopter and $\mathbf{g}_{\mathcal{W}}$ denotes the gravity vector. Two forces act on the quadrotor: the lift force $\mathbf{f}_{\text{prop}}$ generated by the propellers and an aerodynamic force $\mathbf{f}_{\text{aero}}$ that aggregates all other forces, such as aerodynamic drag, dynamic lift and induced drag. The torque is modelled as a sum of four components: the torque generated by the individual propeller thrusts $\boldsymbol{\tau}_{\text{prop}}$, the yaw torque $\boldsymbol{\tau}_{\text{mot}}$ generated by a change in motor speed, an aerodynamic torque $\boldsymbol{\tau}_{\text{aero}}$ that accounts for various aerodynamic effects such as blade flapping and an inertial term $\boldsymbol{\tau}_{\text{iner}}$. The individual components are given as

$$\mathbf{f}_{\text{prop}} = \sum_i \mathbf{f}_i,\ \boldsymbol{\tau}_{\text{prop}} = \sum_i \boldsymbol{\tau}_i + \mathbf{r}_{\text{P},i} \times \mathbf{f}_i, \tag{2}$$

$$\boldsymbol{\tau}_{\text{mot}} = J_{\text{m+p}} \sum_i \boldsymbol{\zeta}_i \dot{\Omega}_i,\ \boldsymbol{\tau}_{\text{iner}} = -\boldsymbol{\omega}_{\mathcal{B}} \times J\boldsymbol{\omega}_{\mathcal{B}} \tag{3}$$

in which $\mathbf{r}_{\text{P},i}$ is the location of propeller i, expressed in the body frame, and $\mathbf{f}_i$ and $\boldsymbol{\tau}_i$ are the forces and torques, respectively, generated by the ith propeller. The axis of rotation of the ith motor is denoted by $\boldsymbol{\zeta}_i$, the combined inertia of the motor and propeller is $J_{\text{m+p}}$ and the derivative of the ith motor speed is $\dot{\Omega}_i$. The individual propellers are modelled using a commonly used quadratic model, which assumes that the lift force and drag torque are proportional to the square of the propeller speed Ω_i:

$$\mathbf{f}_i(\Omega_i) = \begin{bmatrix} 0 & 0 & c_{\text{l}} \cdot \Omega_i^2 \end{bmatrix}^{\top},\quad \boldsymbol{\tau}_i(\Omega_i) = \begin{bmatrix} 0 & 0 & c_{\text{d}} \cdot \Omega_i^2 \end{bmatrix}^{\top} \tag{4}$$

in which c_{l} and c_{d} denote the propeller lift and drag coefficients, respectively.

Aerodynamic forces and torques. The aerodynamic forces and torques are difficult to model with a first-principles approach. We thus use a data-driven model[43]. To maintain the low computational complexity required for large-scale RL training, a grey-box polynomial model is used rather than a neural network. The aerodynamic effects are assumed to primarily depend on the velocity $\mathbf{v}_{\mathcal{B}}$ (in the body frame) and the average squared motor speed $\overline{\Omega^2}$. The aerodynamic forces f_x, f_y and f_z and torques τ_x, τ_y and τ_z are estimated in the body frame. The quantities v_x, v_y and v_z denote the three axial velocity components (in the body frame) and v_{xy} denotes the speed in the (x, y) plane of the quadrotor. On the basis of insights from the underlying physical processes, linear and quadratic combinations of the individual terms are selected. For readability, the coefficients multiplying each summand have been omitted:

$$\begin{aligned}
f_x &\sim v_x + v_x|v_x| + \overline{\Omega^2} + v_x\ \overline{\Omega^2} \\
f_y &\sim v_y + v_y|v_y| + \overline{\Omega^2} + v_y\ \overline{\Omega^2} \\
f_z &\sim v_z + v_z|v_z| + v_{xy} + v_{xy}^2 + v_{xy}\ \overline{\Omega^2} + v_z\ \overline{\Omega^2} + v_{xy}\, v_z\ \overline{\Omega^2} \\
\tau_x &\sim v_y + v_y|v_y| + \overline{\Omega^2} + v_y\ \overline{\Omega^2} + v_y|v_y|\ \overline{\Omega^2} \\
\tau_y &\sim v_x + v_x|v_x| + \overline{\Omega^2} + v_x\ \overline{\Omega^2} + v_x|v_x|\ \overline{\Omega^2} \\
\tau_z &\sim v_x + v_y
\end{aligned}$$

The respective coefficients are then identified from real-world flight data, in which motion capture is used to provide ground-truth forces and torque measurements. We use data from the race track, allowing the dynamics model to fit the track. This is akin to the human pilots' training for days or weeks before the race on the specific track that they will be racing. In our case, the human pilots are given a week of practice on the same track ahead of the competition.

Betaflight low-level controller. To control the quadrotor, the neural network outputs collective thrust and body rates. This control signal is known to combine high agility with good robustness to simulation-to-reality transfer[44]. The predicted collective thrust and body rates are then processed by an onboard low-level controller that computes individual motor commands, which are subsequently translated into analogue voltage signals through an electronic speed controller (ESC) that controls the motors. On the physical vehicle, this low-level proportional–integral–derivative (PID) controller and ESC are implemented using the open-source Betaflight and BLHeli32 firmware[45]. In simulation, we use an accurate model of both the low-level controller and the motor speed controller.

Because the Betaflight PID controller has been optimized for human-piloted flight, it exhibits some peculiarities, which the simulation correctly captures: the reference for the D-term is constantly zero (pure damping), the I-term gets reset when the throttle is cut and, under motor thrust saturation, the body rate control is assigned priority (proportional downscaling of all motor signals to avoid saturation). The gains of the controller used for simulation have been identified from the detailed logs of the Betaflight controller's internal states. The simulation can predict the individual motor commands with less than 1% error.

Battery model and ESC. The low-level controller converts the individual motor commands into a pulse-width modulation (PWM) signal and sends it to the ESC, which controls the motors. Because the ESC does not perform closed-loop control of the motor speeds, the steady-state motor speed $\Omega_{i,\text{ss}}$ for a given PWM motor command cmd_i is a function of the battery voltage. Our simulation thus models the battery voltage using a grey-box battery model[46] that simulates the voltage based on instantaneous power consumption P_{mot}:

$$P_{\text{mot}} = \frac{c_{\text{d}}\Omega^3}{\eta} \tag{5}$$

The battery model[46] then simulates the battery voltage based on this power demand. Given the battery voltage U_{bat} and the individual motor command $u_{\text{cmd},i}$, we use the mapping (again omitting the coefficients multiplying each summand)

$$\Omega_{i,\text{ss}} \sim 1 + U_{\text{bat}} + \sqrt{u_{\text{cmd},i}} + u_{\text{cmd},i} + U_{\text{bat}}\sqrt{u_{\text{cmd},i}} \tag{6}$$

to calculate the corresponding steady-state motor speed $\Omega_{i,\text{ss}}$ required for the dynamics simulation in equation (1). The coefficients have been

文章

方法

四旋翼仿真

四旋翼动力学。为实现大规模训练，我们使用了四旋翼动力学的高保真仿真。本节简要说明该仿真。飞行器的动力学可以写为

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{\mathbf{p}}_{\mathcal{WB}} \\ \dot{\mathbf{q}}_{\mathcal{WB}} \\ \dot{\mathbf{v}}_{\mathcal{W}} \\ \dot{\boldsymbol{\omega}}_{\mathcal{B}} \\ \dot{\Omega} \end{bmatrix} = \begin{bmatrix} \mathbf{v}_{\mathcal{W}} \\ \mathbf{q}_{\mathcal{WB}} \cdot \begin{bmatrix} 0 \\ \boldsymbol{\omega}_{\mathcal{B}}/2 \end{bmatrix} \\ \frac{1}{m}(\mathbf{q}_{\mathcal{WB}} \odot (\mathbf{f}_{\text{prop}} + \mathbf{f}_{\text{aero}})) + \mathbf{g}_{\mathcal{W}} \\ J^{-1}(\boldsymbol{\tau}_{\text{prop}} + \boldsymbol{\tau}_{\text{mot}} + \boldsymbol{\tau}_{\text{aero}} + \boldsymbol{\tau}_{\text{iner}}) \\ \frac{1}{k_{\text{mot}}}(\Omega_{\text{ss}} - \Omega) \end{bmatrix}, \tag{1}$$

其中$\odot$表示四元数旋转，$\mathbf{p}$ $\mathbf{q}$ $\mathbf{v}_{\mathcal{WB}}$ $\mathcal{WB}$ $\mathcal{W}$、、 和$\boldsymbol{\omega}_{\mathcal{B}}$分别表示四旋翼的位置、姿态四元数、惯性速度和机体角速率。电机时间常数为k_{mot}，电机转速$\boldsymbol{\Omega}$和$\boldsymbol{\Omega}_{\text{ss}}$分别为实际电机转速和稳态电机转速。矩阵$J$为四旋翼的惯性，$\mathcal{W}\mathbf{g}$表示重力向量。有两个力作用于四旋翼：由螺旋桨产生的升力$\mathbf{f}_{\text{prop}}$，以及一个气动力$\mathbf{f}_{\text{aero}}$，该力汇总了所有其他力，如气动阻力、动态升力和诱导阻力。力矩被建模为四个分量之和：由各个螺旋桨推力产生的力矩$\boldsymbol{\tau}_{\text{prop}}$、由电机转速变化产生的偏航力矩$\boldsymbol{\tau}_{\text{mot}}$、考虑桨叶挥舞等各种气动效应的气动力矩$\boldsymbol{\tau}_{\text{aero}}$，以及惯性项$\boldsymbol{\tau}_{\text{iner}}$。各分量如下所示

$$\mathbf{f}_{\text{prop}} = \sum_i \mathbf{f}_i,\ \boldsymbol{\tau}_{\text{prop}} = \sum_i \boldsymbol{\tau}_i + \mathbf{r}_{\text{P},i} \times \mathbf{f}_i, \tag{2}$$

$$\boldsymbol{\tau}_{\text{mot}} = J_{\text{m+p}} \sum_i \boldsymbol{\zeta}_i \dot{\Omega}_i,\ \boldsymbol{\tau}_{\text{iner}} = -\boldsymbol{\omega}_{\mathcal{B}} \times J\boldsymbol{\omega}_{\mathcal{B}} \tag{3}$$

其中$\mathbf{r}_{\text{P},i}$是螺旋桨i在机体坐标系中的位置，$\mathbf{f}_i$和$\boldsymbol{\tau}_i$分别是第i个螺旋桨产生的力和力矩。第i个电机的旋转轴记为$\boldsymbol{\zeta}_i$，电机和螺旋桨的合惯性为$J_{\text{m+p}}$，第i个电机转速的导数为$\dot{\Omega}_i$。各个螺旋桨采用常用的二次模型进行建模，该模型假设升力和阻力力矩与螺旋桨转速的平方成正比Ω_i：

$$\mathbf{f}_i(\Omega_i) = \begin{bmatrix} 0 & 0 & c_{\text{l}} \cdot \Omega_i^2 \end{bmatrix}^{\top},\quad \boldsymbol{\tau}_i(\Omega_i) = \begin{bmatrix} 0 & 0 & c_{\text{d}} \cdot \Omega_i^2 \end{bmatrix}^{\top} \tag{4}$$

其中c_{l}和c_{d}分别表示螺旋桨升力系数和阻力系数。

气动力和力矩。气动力和力矩难以用第一性原理方法建模。因此我们使用数据驱动模型[43]。为保持大规模强化学习训练所需的低计算复杂度，采用灰盒多项式模型而非神经网络。假设气动效应主要取决于速度$\mathbf{v}_{\mathcal{B}}$（在机体坐标系中）以及平均电机转速平方$\overline{\Omega^2}$。气动力f_x、f_y和f_z以及力矩τ_x、τ_y和τ_z均在机体坐标系中估计。量v_x、v_y和v_z表示三个轴向速度分量（在机体坐标系中），而$\overline{v_{xy}}$表示四旋翼在(x, y)平面内的速度。基于对底层物理过程的洞察，选择各项的线性与二次组合。为便于阅读，乘以每个求和项的系数已被省略：

$$\begin{aligned}
f_x &\sim v_x + v_x|v_x| + \overline{\Omega^2} + v_x\ \overline{\Omega^2} \\
f_y &\sim v_y + v_y|v_y| + \overline{\Omega^2} + v_y\ \overline{\Omega^2} \\
f_z &\sim v_z + v_z|v_z| + v_{xy} + v_{xy}^2 + v_{xy}\ \overline{\Omega^2} + v_z\ \overline{\Omega^2} + v_{xy}\, v_z\ \overline{\Omega^2} \\
\tau_x &\sim v_y + v_y|v_y| + \overline{\Omega^2} + v_y\ \overline{\Omega^2} + v_y|v_y|\ \overline{\Omega^2} \\
\tau_y &\sim v_x + v_x|v_x| + \overline{\Omega^2} + v_x\ \overline{\Omega^2} + v_x|v_x|\ \overline{\Omega^2} \\
\tau_z &\sim v_x + v_y
\end{aligned}$$

随后从真实世界飞行数据中辨识各自的系数，其中使用动作捕捉来提供真值力和力矩测量。我们使用来自赛道的数据，使动力学模型能够拟合该赛道。这类似于人类飞行员在比赛前数天或数周在将要比赛的特定赛道上进行训练。在我们的案例中，人类飞行员在比赛前获得了一周在同一赛道上的练习。

Betaflight 低级控制器。为控制四旋翼，神经网络输出集体推力和机体角速率。已知这种控制信号兼具高敏捷性和对仿真到现实迁移的良好鲁棒性[44]。随后，预测的集体推力和机体角速率由机载低级控制器处理，该控制器计算出各个电机指令，这些指令随后通过控制电机的电子调速器（ESC）转换为模拟电压信号。在物理飞行器上，该低级比例-积分-微分（PID）控制器和 ESC 使用开源的 Betaflight 和 BLHeli32 固件实现[45]。在仿真中，我们使用低级控制器和电机转速控制器的精确模型。

由于Betaflight PID控制器针对人工驾驶飞行进行了优化，它表现出一些特殊性，仿真正确地捕捉到了这些特性：D项的参考值始终为零（纯阻尼），油门切断时I项会被重置，并且在电机推力饱和时，机体角速率控制被赋予优先级（对所有电机信号进行比例降幅以避免饱和）。用于仿真的控制器增益已从Betaflight控制器内部状态的详细日志中辨识得到。仿真能够以低于1%的误差预测各个电机指令。

电池模型与电子调速器。低级控制器将各个电机指令转换为脉宽调制（PWM）信号并发送至电子调速器，由电子调速器控制电机。由于电子调速器不对电机转速执行闭环控制，稳态电机转速$\Omega_{i,\text{ ss}}$对于给定的PWM电机指令cmd_i是电池电压的函数。因此，我们的仿真使用灰箱电池模型对电池电压进行建模[46]，该模型基于瞬时功耗P_{mot}来模拟电压：

$$P_{\text{mot}} = \frac{c_{\text{d}}\Omega^3}{\eta} \tag{5}$$

电池模型[46]然后根据该功率需求模拟电池电压。给定电池电压U_{bat}和各电机指令$u_{\text{cmd},\ i}$，我们使用该映射（再次省略了与各求和项相乘的系数）

$$\Omega_{i,\text{ss}} \sim 1 + U_{\text{bat}} + \sqrt{u_{\text{cmd},i}} + u_{\text{cmd},i} + U_{\text{bat}}\sqrt{u_{\text{cmd},i}} \tag{6}$$

以计算相应的稳态电机转速$\Omega_{i,\text{ ss}}$这是方程（1）中动力学仿真所需的。这些系数已

identified from Betaflight logs containing measurements of all involved quantities. Together with the model of the low-level controller, this enables the simulator to correctly translate an action in the form of collective thrust and body rates to desired motor speeds Ω_{ss} in equation (1).

Policy training

We train deep neural control policies that directly map observations $\mathbf{o}_t$ in the form of platform state and next gate observation to control actions $\mathbf{u}_t$ in the form of mass-normalized collective thrust and body rates[44]. The control policies are trained using model-free RL in simulation.

Training algorithm. Training is performed using proximal policy optimization[31]. This actor-critic approach requires jointly optimizing two neural networks during training: the policy network, which maps observations to actions, and the value network, which serves as the 'critic' and evaluates actions taken by the policy. After training, only the policy network is deployed on the robot.

Observations, actions and rewards. An observation $\mathbf{o}_t \in \mathbb{R}^{31}$ obtained from the environment at time t consists of: (1) an estimate of the current robot state; (2) the relative pose of the next gate to be passed on the track layout; and (3) the action applied in the previous step. Specifically, the estimate of the robot state contains the position of the platform, its velocity and attitude represented by a rotation matrix, resulting in a vector in $\mathbb{R}^{15}$. Although the simulation uses quaternions internally, we use a rotation matrix to represent attitude to avoid ambiguities[47]. The relative pose of the next gate is encoded by providing the relative position of the four gate corners with respect to the vehicle, resulting in a vector in $\mathbb{R}^{12}$. All observations are normalized before being passed to the network. Because the value network is only used during training time, it can access privileged information about the environment that is not accessible to the policy[48]. This privileged information is concatenated with other inputs to the policy network and contains the exact position, orientation and velocity of the robot.

For each observation $\mathbf{o}_t$, the policy network produces an action $\mathbf{a}_t \in \mathbb{R}^4$ in the form of desired mass-normalized collective thrust and body rates.

We use a dense shaped reward formulation to learn the task of perception-aware autonomous drone racing. The reward r_t at time step t is given by

$$r_t = r_t^{\text{prog}} + r_t^{\text{perc}} + r_t^{\text{cmd}} - r_t^{\text{crash}} \tag{7}$$

in which r^{prog} rewards progress towards the next gate[35], r^{perc} encodes perception awareness by adjusting the attitude of the vehicle such that the optical axis of the camera points towards the centre of the next gate, r^{cmd} rewards smooth actions and r^{crash} is a binary penalty that is only active when colliding with a gate or when the platform leaves a predefined bounding box. If r^{crash} is triggered, the training episode ends.

Specifically, the reward terms are

$$\begin{aligned} r_t^{\text{prog}} &= \lambda_1[d_{t-1}^{\text{Gate}} - d_t^{\text{Gate}}] \\ r_t^{\text{perc}} &= \lambda_2 \exp[\lambda_3 \cdot \delta_{\text{cam}}^4] \end{aligned} \tag{8}$$

$$r_t^{\text{cmd}} = \lambda_4 \mathbf{a}_t^{\omega} + \lambda_5 \|\mathbf{a}_t - \mathbf{a}_{t-1}\|^2 \tag{9}$$

$$r_t^{\text{crash}} = \begin{cases} 5.0, & \text{if } p_z < 0 \text{ or in collision with gate} \\ 0, & \text{otherwise} \end{cases}$$

in which d_t^{Gate} denotes the distance from the centre of mass of the vehicle to the centre of the next gate at time step t, δ_{cam} represents the angle between the optical axis of the camera and the centre of the next gate and $\mathbf{a}_t^{\omega}$ are the commanded body rates. The hyperparameters $\lambda_1, \ldots, \lambda_5$ balance different terms (Extended Data Table 1a).

Training details. Data collection is performed by simulating 100 agents in parallel that interact with the environment in episodes of 1,500 steps. At each environment reset, every agent is initialized at a random gate on the track, with bounded perturbation around a state previously observed when passing this gate. In contrast to previous work[44,49,50], we do not perform randomization of the platform dynamics at training time. Instead, we perform fine-tuning based on real-world data. The training environment is implemented using TensorFlow Agents[51]. The policy network and the value network are both represented by two-layer perceptrons with 128 nodes in each layer and LeakyReLU activations with a negative slope of 0.2. Network parameters are optimized using the Adam optimizer with learning rate 3×10^{-4} for both the policy network and the value network.

Policies are trained for a total of 1×10^8 environment interactions, which takes 50 min on a workstation (i9 12900K, RTX 3090, 32 GB RAM DDR5). Fine-tuning is performed for 2×10^7 environment interactions.

Residual model identification

We perform fine-tuning of the original policy based on a small amount of data collected in the real world. Specifically, we collect three full rollouts in the real world, corresponding to approximately 50 s of flight time. We fine-tune the policy by identifying residual observations and residual dynamics, which are then used for training in simulation. During this fine-tuning phase, only the weights of the control policy are updated, whereas the weights of the gate-detection network are kept constant.

Residual observation model. Navigating at high speeds results in substantial motion blur, which can lead to a loss of tracked visual features and severe drift in linear odometry estimates. We fine-tune policies with an odometry model that is identified from only a handful of trials recorded in the real world. To model the drift in odometry, we use Gaussian processes[36], as they allow fitting a posterior distribution of odometry perturbations, from which we can sample temporally consistent realizations.

Specifically, the Gaussian process model fits residual position, velocity and attitude as a function of the ground-truth robot state. The observation residuals are identified by comparing the observed visual–inertial odometry (VIO) estimates during a real-world rollout with the ground-truth platform states, which are obtained from an external motion-tracking system.

We treat each dimension of the observation separately, effectively fitting a set of nine 1D Gaussian processes to the observation residuals. We use a mixture of radial basis function kernels

$$\kappa(\mathbf{z}_i, \mathbf{z}_j) = \sigma_f^2 \exp\left(-\frac{1}{2}(\mathbf{z}_i - \mathbf{z}_j)^{\top} L^{-2} (\mathbf{z}_i - \mathbf{z}_j)\right) + \sigma_n^2 \tag{10}$$

in which L is the diagonal length scale matrix and σ_f and σ_n represent the data and prior noise variance, respectively, and $\mathbf{z}_i$ and $\mathbf{z}_j$ represent data features. The kernel hyperparameters are optimized by maximizing the log marginal likelihood. After kernel hyperparameter optimization, we sample new realizations from the posterior distribution that are then used during fine-tuning of the policy. Extended Data Fig. 1 illustrates the residual observations in position, velocity and attitude in real-world rollouts, as well as 100 sampled realizations from the Gaussian process model.

Residual dynamics model. We use a residual model to complement the simulated robot dynamics[52]. Specifically, we identify residual accelerations as a function of the platform state $\mathbf{s}$ and the commanded mass-normalized collective thrust c:

从包含所有相关量测量的Betaflight日志中辨识得到。结合低级控制器模型，这使模拟器能够正确地将以集体推力和机体角速率形式表示的动作转换为期望电机转速Ω_{ss}，见方程（1）。

策略训练

我们训练深度神经控制策略，其直接将观测$\mathbf{o}_t$以平台状态和下一门观测的形式映射到控制动作$\mathbf{u}_t$，动作以质量归一化集体推力和机体角速率的形式表示[44]。控制策略在仿真中使用无模型强化学习进行训练。

训练算法。训练使用近端策略优化[31]进行。这种演员-评论家方法需要在训练期间联合优化两个神经网络：策略网络，将观测映射到动作；以及价值网络，充当“评论家”并评估策略所采取的动作。训练后，仅策略网络部署在机器人上。

观测、动作和奖励。在时间t从环境中获得的观测$\mathbf{o}_t \in \mathbb{R}^{31}$包括：(1) 当前机器人状态的估计；(2) 赛道上需要通过的下一个门的相对位姿；以及 (3) 上一步施加的动作。具体地，机器人状态的估计包含平台的位置、其速度以及用旋转矩阵表示的态度，从而得到$\mathbb{R}^{15}$中的一个向量。尽管仿真内部使用四元数，我们使用旋转矩阵来表示态度以避免歧义[47]。下一个门的相对位姿通过提供四个门角相对于飞行器的相对位置来编码，从而得到$\mathbb{R}^{12}$中的一个向量。所有观测在传递给网络之前都被归一化。由于价值网络仅在训练时使用，它可以访问策略无法访问的环境特权信息[48]。此特权信息与策略网络的其他输入拼接，并包含机器人的精确位置、朝向和速度。

对于每个观测 $\mathbf{o}_t$，策略网络产生一个 $\text{action}_t \in \mathbb{R}^4$ $\mathbf{a}$，其形式为期望的质量归一化集体推力和机体角速率。

我们使用密集塑形奖励公式来学习感知感知自主无人机竞速任务。奖励 r_t 在时间步 t 由下式给出

$$r_t = r_t^{\text{prog}} + r_t^{\text{perc}} + r_t^{\text{cmd}} - r_t^{\text{crash}} \tag{7}$$

其中 r^{prog} 奖励朝向下一门的进度[35]，r^{perc} 通过调整飞行器的姿态，使摄像头的光轴指向下一门中心，从而编码感知意识，r^{cmd} 奖励平滑动作，并且 r^{crash} 是一个二元惩罚，仅在与门碰撞或平台离开预定义边界框时激活。如果 r^{crash} 被触发，训练回合结束。

具体而言，奖励项为

$$\begin{aligned} r_t^{\text{prog}} &= \lambda_1[d_{t-1}^{\text{Gate}} - d_t^{\text{Gate}}] \\ r_t^{\text{perc}} &= \lambda_2 \exp[\lambda_3 \cdot \delta_{\text{cam}}^4] \end{aligned} \tag{8}$$

$$r_t^{\text{cmd}} = \lambda_4 \mathbf{a}_t^{\omega} + \lambda_5 \|\mathbf{a}_t - \mathbf{a}_{t-1}\|^2 \tag{9}$$

$$r_t^{\text{crash}} = \begin{cases} 5.0, & \text{if } p_z < 0 \text{ or in collision with gate} \\ 0, & \text{otherwise} \end{cases}$$

其中 d_t^{Gate} 表示在时间步 t 时，从飞行器质心到下一门中心的距离，δ_{cam} 表示摄像头光轴与下一门中心之间的角度

而 $_t{}^{\omega}\mathbf{a}$ 是指令机体角速率。超参数 $\lambda_1, \ldots, \lambda_5$用于平衡不同的项（扩展数据表1a）。

训练细节。数据收集通过并行模拟100个智能体完成，这些智能体以1,500步的回合与环境交互。每次环境重置时，每个智能体在赛道上的随机门处初始化，并在先前通过该门时观测到的状态附近有界扰动。与先前工作相比[44,49,50],我们在训练时不执行平台动力学的随机化。相反，我们基于真实世界数据进行微调。训练环境使用TensorFlow Agents实现[51]。策略网络和价值网络均由两层感知机表示，每层128个节点，并使用负斜率为0.2的LeakyReLU激活函数。网络参数使用Adam优化器优化，学习率为 3×10^{-4}，用于策略网络和价值网络。

策略被训练总共进行 1×10^8 次环境交互，这在工作站（i9 12900K、RTX 3090、32 GB RAM DDR5）上需要 50 分钟。微调则进行 2×10^7 次环境交互。

残差模型辨识

我们基于在真实世界中收集的少量数据，对原始策略进行微调。具体而言，我们在真实世界中采集三次完整 rollout，对应约 50 秒飞行时间。我们通过辨识残差观测和残差动力学来微调策略，随后将其用于仿真训练。在此微调阶段，仅更新控制策略的权重，而门检测网络的权重保持不变。

残差观测模型。高速飞行会导致显著的运动模糊，这可能导致跟踪到的视觉特征丢失以及线性里程计估计的严重漂移。我们使用仅从真实世界中记录的少量试验中辨识出的里程计模型来微调策略。为了对里程计漂移建模，我们使用高斯过程[36]，因为它们允许拟合里程计扰动的后验分布，从中我们可以采样时间上一致的实现。

具体而言，高斯过程模型将残差位置、速度和姿态拟合为真值机器人状态的函数。通过将真实世界滚动中观测到的视觉惯性里程计（VIO）估计值与从外部运动跟踪系统获得的真值平台状态进行比较，来识别观测残差。

我们分别处理观测的每个维度，从而有效地将一组九个一维高斯过程拟合到观测残差。我们使用径向基函数核的混合

$$\kappa(\mathbf{z}_i, \mathbf{z}_j) = \sigma_f^2 \exp\left(-\frac{1}{2}(\mathbf{z}_i - \mathbf{z}_j)^{\top} L^{-2} (\mathbf{z}_i - \mathbf{z}_j)\right) + \sigma_n^2 \tag{10}$$

其中$\boldsymbol{L}$是对角长度尺度矩阵，σ_f和σ_n分别表示数据和先验噪声方差，$\mathbf{z}_i$和$\mathbf{z}_j$表示数据特征。核超参数通过最大化对数边际似然来优化。在核超参数优化之后，我们从后验分布中采样新的实现，这些实现随后用于策略的微调。扩展数据图1展示了真实世界滚动中位置、速度和姿态的残差观测，以及从高斯过程模型中采样的100个实现。

残差动力学模型。我们使用残差模型来补充仿真机器人动力学[52]。具体而言，我们将残差加速度辨识为平台状态$\mathbf{s}$和指令质量归一化集体推力c的函数：

$$\mathbf{a}_{\text{res}} = \text{KNN}(\mathbf{s}, c) \tag{11}$$

We use k-nearest neighbour regression with $k = 5$. The size of the dataset used for residual dynamics model identification depends on the track layout and ranges between 800 and 1,000 samples for the track layout used in this work.

Gate detection

To correct for drift accumulated by the VIO pipeline, the gates are used as distinct landmarks for relative localization. Specifically, gates are detected in the onboard camera view by segmenting gate corners[26]. The greyscale images provided by the Intel RealSense Tracking Camera T265 are used as input images for the gate detector. The architecture of the segmentation network is a six-level U-Net[53] with (8, 16, 16, 16, 16, 16) convolutional filters of size (3, 3, 3, 5, 7, 7) per level and a final extra layer operating on the output of the U-Net containing 12 filters. As the activation function, LeakyReLU with $\alpha = 0.01$ is used. For deployment on the NVIDIA Jetson TX2, the network is ported to TensorRT. To optimize memory footprint and computation time, inference is performed in half-precision mode (FP16) and images are downsampled to size 384 × 384 before being fed to the network. One forward pass through the network takes 40 ms on the NVIDIA Jetson TX2.

VIO drift estimation

The odometry estimates from the VIO pipeline[54] exhibit substantial drift during high-speed flight. We use gate detection to stabilize the pose estimates produced by VIO. The gate detector outputs the coordinates of the corners of all visible gates. A relative pose is first estimated for all predicted gates using infinitesimal plane-based pose estimation (IPPE)[34]. Given this relative pose estimate, each gate observation is assigned to the closest gate in the known track layout, thus yielding a pose estimate for the drone.

Owing to the low frequency of the gate detections and the high quality of the VIO orientation estimate, we only refine the translational components of the VIO measurements. We estimate and correct for the drift of the VIO pipeline using a Kalman filter that estimates the translational drift $\mathbf{p}_\text{d}$ (position offset) and its derivative, the drift velocity $\mathbf{v}_\text{d}$. The drift correction is performed by subtracting the estimated drift states $\mathbf{p}_\text{d}$ and $\mathbf{v}_\text{d}$ from the corresponding VIO estimates. The Kalman filter state $\mathbf{x}$ is given by $\mathbf{x} = [\mathbf{p}_\text{d}^\top, \mathbf{v}_\text{d}^\top]^\top \in \mathbb{R}^6$.

The state $\mathbf{x}$ and covariance P updates are given by:

$$\mathbf{x}_{k+1} = F\mathbf{x}_k, \quad P_{k+1} = FP_kF^\top + Q, \tag{12}$$

$$F = \begin{bmatrix} \mathbb{I}^{3\times3} & \mathrm{d}t\,\mathbb{I}^{3\times3} \\ 0^{3\times3} & \mathbb{I}^{3\times3} \end{bmatrix}, \quad Q = \begin{bmatrix} \sigma_\text{pos}\mathbb{I}^{3\times3} & 0^{3\times3} \\ 0^{3\times3} & \sigma_\text{vel}\mathbb{I}^{3\times3} \end{bmatrix}. \tag{13}$$

On the basis of measurements, the process noise is set to $\sigma_\text{pos} = 0.05$ and $\sigma_\text{vel} = 0.1$. The filter state and covariance are initialized to zero. For each measurement $\mathbf{z}_k$ (pose estimate from a gate detection), the predicted VIO drift $\mathbf{x}_k^-$ is corrected to the estimate $\mathbf{x}_k^+$ according to the Kalman filter equations:

$$\begin{aligned} K_k &= P_k^- H_k^\top (H_k P_k^- H_k^\top + R)^{-1}, \\ \mathbf{x}_k^+ &= \mathbf{x}_k^- + K_k(\mathbf{z}_k - H(\mathbf{x}_k^-)), \\ P_k^+ &= (I - K_k H_k) P_k^-, \end{aligned} \tag{14}$$

in which K_k is the Kalman gain, R is the measurement covariance and H_k is the measurement matrix. If several gates have been detected in a single camera frame, all relative pose estimates are stacked and processed in the same Kalman filter update step. The main source of measurement error is the uncertainty in the gate-corner detection of the network. This error in the image plane results in a pose error when IPPE is applied. We opted for a sampling-based approach to estimate the pose error from the known average gate-corner-detection uncertainty. For each gate, the IPPE algorithm is applied to the nominal gate observation as well as to 20 perturbed gate-corner estimates. The resulting distribution of pose estimates is then used to approximate the measurement covariance R of the gate observation.

Simulation results

Reaching champion-level performance in autonomous drone racing requires overcoming two challenges: imperfect perception and incomplete models of the system's dynamics. In controlled experiments in simulation, we assess the robustness of our approach to both of these challenges. To this end, we evaluate performance in a racing task when deployed in four different settings. In setting (1), we simulate a simplistic quadrotor model with access to ground-truth state observations. In setting (2), we replace the ground-truth state observations with noisy observations identified from real-world flights. These noisy observations are generated by sampling one realization from the residual observation model and are independent of the perception awareness of the deployed controller. Settings (3) and (4) share the observation models with the previous two settings, respectively, but replace the simplistic dynamics model with more accurate aerodynamical simulation[43]. These four settings allow controlled assessment of the sensitivity of the approach to changes in the dynamics and the observation fidelity.

In all four settings, we benchmark our approach against the following baselines: zero-shot, domain randomization and time-optimal. The zero-shot baseline represents a learning-based racing policy[35] trained using model-free RL that is deployed zero-shot from the training domain to the test domain. The training domain of the policy is equal to experimental setting (1), that is, idealized dynamics and ground-truth observations. Domain randomization extends the learning strategy from the zero-shot baseline by randomizing observations and dynamics properties to increase robustness. The time-optimal baseline uses a precomputed time-optimal trajectory[28] that is tracked using an MPC controller. This approach has shown the best performance in comparison with other model-based methods for time-optimal flight[55,56]. The dynamics model used by the trajectory generation and the MPC controller matches the simulated dynamics of experimental setting (1).

Performance is assessed by evaluating the fastest lap time, the average and minimum observed gate margin of successfully passed gates and the percentage of track successfully completed. The gate margin metric measures the distance between the drone and the closest point on the gate when crossing the gate plane. A high gate margin indicates that the quadrotor passed close to the centre of the gate. Leaving a smaller gate margin can increase speed but can also increase the risk of collision or missing the gate. Any lap that results in a crash is not considered valid.

The results are summarized in Extended Data Table 1c. All approaches manage to successfully complete the task when deployed in idealized dynamics and ground-truth observations, with the time-optimal baseline yielding the lowest lap time. When deployed in settings that feature domain shift, either in the dynamics or the observations, the performance of all baselines collapses and none of the three baselines are able to complete even a single lap. This performance drop is exhibited by both learning-based and traditional approaches. By contrast, our approach, which features empirical models of dynamics and observation noise, succeeds in all deployment settings, with small increases in lap time.

The key feature that enables our approach to succeed across deployment regimes is the use of an empirical model of dynamics and observation noise, estimated from real-world data. A comparison between an approach that has access to such data and approaches that do not is not entirely fair. For that reason, we also benchmark the performance of all baseline approaches when having access to the same real-world data used by our approach. Specifically, we

$$\mathbf{a}_{\text{res}} = \text{KNN}(\mathbf{s}, c) \tag{11}$$

我们使用 k-最近邻回归，其中 $k = 5$。用于残差动力学模型辨识的数据集大小取决于赛道布局，对于本工作中使用的赛道布局，其范围在800到1,000个样本之间。

门检测

为校正VIO管线累积的漂移，门被用作相对定位的独特地标。具体而言，通过分割门角在机载摄像头视图中检测门[26]。Intel RealSense追踪相机T265提供的灰度图像用作门检测器的输入图像。分割网络的架构是六级U-Net[53]，每级具有(8, 16, 16, 16, 16, 16)个大小为(3, 3, 3, 5, 7, 7)的卷积滤波器，以及一个对U-Net输出进行操作的最终额外层，包含12个滤波器。激活函数使用LeakyReLU，$\alpha = 0.01$。为部署在NVIDIA Jetson TX2上，网络被移植到TensorRT。为优化内存占用和计算时间，推理在半精度模式(FP16)下进行，图像在输入网络前被下采样至384 × 384大小。在NVIDIA Jetson TX2上，一次前向传播需要40 ms。

VIO漂移估计

来自VIO管线的里程计估计[54]在高速飞行期间表现出显著的漂移。我们使用门检测来稳定VIO产生的位姿估计。门检测器输出所有可见门的角点坐标。首先使用基于无穷小平面的位姿估计（IPPE）对所有预测的门估计相对位姿[34]。给定该相对位姿估计，每个门观测被分配给已知赛道布局中最近的门，从而得到无人机的位姿估计。

由于门检测频率较低且VIO方向估计质量较高，我们仅对VIO测量的平移分量进行修正。我们使用卡尔曼滤波器估计并校正VIO管线的漂移，该滤波器估计平移漂移$\mathbf{p}_\text{d}$（位置偏移）及其导数，即漂移速度$\mathbf{v}_\text{d}$。漂移校正通过从相应的VIO估计中减去估计的漂移状态$\mathbf{p}_\text{d}$和$\mathbf{v}_\text{d}$来执行。卡尔曼滤波器状态$\mathbf{x}$由下式给出$\mathbb{R}\mathbf{x}\ \mathbf{p}\ \mathbf{v} = [\ ^\top_\text{d},\ ^\top]^\top \in_\text{d}\ ^6$。

状态 $\mathbf{x}$ 和协方差 P 的更新由下式给出：

$$\mathbf{x}_{k+1} = F\mathbf{x}_k, \quad P_{k+1} = FP_kF^\top + Q, \tag{12}$$

$$F = \begin{bmatrix} \mathbb{I}^{3\times3} & \mathrm{d}t\,\mathbb{I}^{3\times3} \\ 0^{3\times3} & \mathbb{I}^{3\times3} \end{bmatrix}, \quad Q = \begin{bmatrix} \sigma_\text{pos}\mathbb{I}^{3\times3} & 0^{3\times3} \\ 0^{3\times3} & \sigma_\text{vel}\mathbb{I}^{3\times3} \end{bmatrix}. \tag{13}$$

根据测量结果，过程噪声设置为 $\sigma_\text{pos} = 0.05$ 和 $\sigma_\text{vel} = 0.1$。滤波器状态和协方差初始化为零。对于每次测量 $\mathbf{z}_k$（来自门检测的位姿估计），预测的 VIO 漂移 $\mathbf{x}_k^-$ 根据卡尔曼滤波器方程被校正为估计值 $\mathbf{x}_k^+$：

$$\begin{aligned} K_k &= P_k^- H_k^\top (H_k P_k^- H_k^\top + R)^{-1}, \\ \mathbf{x}_k^+ &= \mathbf{x}_k^- + K_k(\mathbf{z}_k - H(\mathbf{x}_k^-)), \\ P_k^+ &= (I - K_k H_k) P_k^-, \end{aligned} \tag{14}$$

其中 K_k 是卡尔曼增益，R 是测量协方差，H_k 是测量矩阵。如果在单个相机帧中检测到多个门，则将所有相对位姿估计堆叠起来，并在同一个卡尔曼滤波器更新步骤中处理。测量误差的主要来源是网络门角点检测中的不确定性。图像平面中的这一误差在应用 IPPE 时会导致位姿误差。

我们选择了一种基于采样的方法，根据已知的平均门角点检测不确定性来估计位姿误差。对于每个门，IPPE 算法既应用于标称门观测，也应用于 20 个扰动后的门角点估计。然后使用所得的位姿估计分布来近似门观测的测量协方差 R。

仿真结果

在自主无人机竞速中达到冠军级表现需要克服两个挑战：不完美感知和系统动力学的不完整模型。在仿真中的受控实验里，我们评估我们的方法对这两个挑战的鲁棒性。为此，我们评估在四种不同设置下部署时竞速任务中的性能。在设置（1）中，我们仿真一个简化四旋翼模型，并访问真值状态观测。在设置（2）中，我们用从真实世界飞行中识别出的噪声观测替换真值状态观测。这些噪声观测通过从残差观测模型中采样一个实现来生成，并且与所部署控制器的感知意识无关。设置（3）和（4）分别与前两个设置共享观测模型，但将简化动力学模型替换为更准确的空气动力学仿真[43]。这四种设置允许对方法对动力学变化和观测保真度变化的敏感性进行受控评估。

在所有四种设置中，我们将我们的方法与以下基线进行基准比较：零样本、域随机化和时间最优。零样本基线代表一个基于学习的竞速策略[35]，该策略使用无模型强化学习训练，并从训练域零样本部署到测试域。该策略的训练域等于实验设置（1），即理想化动力学和真值观测。域随机化通过随机化观测和动力学属性来扩展零样本基线的学习策略，以提高鲁棒性。时间最优基线使用预计算的时间最优轨迹[28]，并通过MPC控制器进行跟踪。与其他基于模型的时间最优飞行方法相比，该方法已展现出最佳性能[55,56]。轨迹生成和MPC控制器所使用的动力学模型与实验设置（1）中的模拟动力学相匹配。

性能评估基于最快圈速、成功通过门的平均和最小观测门裕度，以及成功完成的赛道百分比。门裕度度量的是无人机穿越门平面时与门上最近点之间的距离。较高的门裕度表明四旋翼飞行器靠近门的中心通过。保留较小的门裕度可以提高速度，但也会增加碰撞或错过门的风险。任何导致坠毁的圈均视为无效。

结果总结于扩展数据表1c。当部署于理想化动力学和真值观测时，所有方法均能成功完成任务，其中时间最优基线取得了最低圈速。当部署于存在域偏移的设置中时，无论是动力学还是观测中的域偏移，所有基线的性能均崩溃，三个基线均无法完成哪怕一圈。这种性能下降在学习型方法和传统方法中均有体现。相比之下，我们的方法以动力学的经验模型和观测噪声为特征，在所有部署设置中均取得成功，圈速仅有小幅增加。

使我们的方法能够在各种部署条件下成功的关键特征是使用从真实世界数据估计得到的动力学与观测噪声的经验模型。能够访问此类数据的方法与无法访问此类数据的方法之间的比较并不完全公平。因此，我们还在所有基线方法能够访问与我们方法所用相同的真实世界数据时，对其性能进行基准测试。具体而言，我们

compare the performance in experimental setting (2), which features the idealized dynamics model but noisy perception. All baseline approaches are provided with the predictions of the same Gaussian process model that we use to characterize observation noise. The results are summarized in Extended Data Table 1b. All baselines benefit from the more realistic observations, yielding higher completion rates. Nevertheless, our approach is the only one that reliably completes the entire track. As well as the predictions of the observation noise model, our approach also takes into account the uncertainty of the model. For an in-depth comparison of the performance of RL versus optimal control in controlled experiments, we refer the reader to ref. 57.

Fine-tuning for several iterations

We investigate the extent of variations in behaviour across iterations. The findings of our analysis reveal that subsequent fine-tuning operations result in negligible enhancements in performance and alterations in behaviour (Extended Data Fig. 2).

In the following, we provide more details on this investigation. We start by enumerating the fine-tuning steps to provide the necessary notation:

1. Train policy-0 in simulation.
2. Deploy policy-0 in the real world. The policy operates on ground-truth data from a motion-capture system.
3. Identify residuals observed by policy-0 in the real world.
4. Train policy-1 by fine-tuning policy-0 on the identified residuals.
5. Deploy policy-1 in the real world. The policy operates only on onboard sensory measurements.
6. Identify residuals observed by policy-1 in the real world.
7. Train policy-2 by fine-tuning policy-1 on the identified residuals.

We compare the performance of policy-1 and policy-2 in simulation after fine-tuning on their respective residuals. The results are illustrated in Extended Data Fig. 2. We observe that the difference in distance from gate centres, which is a metric for the safety of the policy, is 0.09 ± 0.08 m. Furthermore, the difference in the time taken to complete a single lap is 0.02 ± 0.02 s. Note that this lap-time difference is substantially smaller than the difference between the single-lap completion times of Swift and the human pilots (0.16 s).

Drone hardware configuration

The quadrotors used by the human pilots and Swift have the same weight, shape and propulsion. The platform design is based on the Agilicious framework[58]. Each vehicle has a weight of 870 g and can produce a maximum static thrust of approximately 35 N, which results in a static thrust-to-weight ratio of 4.1. The base of each platform consists of an Armattan Chameleon 6″ main frame that is equipped with T-Motor Velox 2306 motors and 5″, three-bladed propellers. An NVIDIA Jetson TX2 accompanied by a Connect Tech Quasar carrier board provides the main compute resource for the autonomous drones, featuring a six-core CPU running at 2 GHz and a dedicated GPU with 256 CUDA cores running at 1.3 GHz. Although forward passes of the gate-detection network are performed on the GPU, the racing policy is evaluated on the CPU, with one inference pass taking 8 ms. The autonomous drones carry an Intel RealSense Tracking Camera T265 that provides VIO estimates[59] at 100 Hz that are fed by USB to the NVIDIA Jetson TX2. The human-piloted drones carry neither a Jetson computer nor a RealSense camera and are instead equipped with a corresponding ballast weight. Control commands in the form of collective thrust and body rates produced by the human pilots or Swift are sent to a commercial flight controller, which runs on an STM32 processor operating at 216 MHz. The flight controller is running Betaflight, an open-source flight-control software[45].

Human pilot impressions

The following quotes convey the impressions of the three human champions who raced against Swift.

Alex Vanover:

- These races will be decided at the split S, it is the most challenging part of the track.
- This was the best race! I was so close to the autonomous drone, I could really feel the turbulence when trying to keep up with it.

Thomas Bitmatta:

- The possibilities are endless, this is the start of something that could change the whole world. On the flip side, I'm a racer, I don't want anything to be faster than me.
- As you fly faster, you trade off precision for speed.
- It's inspiring to see the potential of what drones are actually capable of. Soon, the AI drone could even be used as a training tool to understand what would be possible.

Marvin Schaepper:

- It feels different racing against a machine, because you know that the machine doesn't get tired.

Research ethics

The study has been conducted in accordance with the Declaration of Helsinki. The study protocol is exempt from review by an ethics committee according to the rules and regulations of the University of Zurich, because no health-related data has been collected. The participants gave their written informed consent before participating in the study.

Data availability

All (other) data needed to evaluate the conclusions in the paper are present in the paper or the extended data. Motion-capture recordings of the race events with accompanying analysis code can be found in the file 'racing_data.zip' on Zenodo at https://doi.org/10.5281/zenodo.7955278.

Code availability

Pseudocode for Swift detailing the training process and algorithms can be found in the file 'pseudocode.zip' on Zenodo at https://doi.org/10.5281/zenodo.7955278. To safeguard against potential misuse, the full source code associated with this research will not be made publicly available.

43. Bauersfeld, L., Kaufmann, E., Foehn, P., Sun, S. & Scaramuzza, D. in *Proc. Robotics: Science and Systems XVII* 42 (Robotics: Science and Systems Foundation, 2021).
44. Kaufmann, E., Bauersfeld, L. & Scaramuzza, D. in *Proc. 2022 International Conference on Robotics and Automation (ICRA)* 10504–10510 (IEEE, 2022).
45. The Betaflight Open Source Flight Controller Firmware Project. Betaflight. https://github.com/betaflight/betaflight (2022).
46. Bauersfeld, L. & Scaramuzza, D. Range, endurance, and optimal speed estimates for multicopters. *IEEE Robot. Autom. Lett.* **7**, 2953–2960 (2022).
47. Zhou, Y., Barnes, C., Lu, J., Yang, J. & Li, H. in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* 5745–5753 (IEEE, 2019).
48. Pinto, L., Andrychowicz, M., Welinder, P., Zaremba, W. & Abbeel, P. in *Proc. Robotics: Science and Systems XIV* (MIT Press Journals, 2018).
49. Molchanov, A. et al. in *Proc. 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* 59–66 (IEEE, 2019).
50. Andrychowicz, O. M. et al. Learning dexterous in-hand manipulation. *Int. J. Robot. Res.* **39**, 3–20 (2020).
51. Guadarrama, S. et al. TF-Agents: a library for reinforcement learning in TensorFlow. https://github.com/tensorflow/agents (2018).
52. Torrente, G., Kaufmann, E., Foehn, P. & Scaramuzza, D. Data-driven MPC for quadrotors. *IEEE Robot. Autom. Lett.* **6**, 3769–3776 (2021).
53. Ronneberger, O., Fischer, P. & Brox, T. in *Proc. International Conference on Medical Image Computing and Computer-assisted Intervention* 234–241 (Springer, 2015).
54. Intel RealSense T265 series product family. https://www.intelrealsense.com/wp-content/uploads/2019/09/Intel_RealSense_Tracking_Camera_Datasheet_Rev004_release.pdf (2019).
55. Ryou, G., Tal, E. & Karaman, S. Multi-fidelity black-box optimization for time-optimal quadrotor maneuvers. *Int. J. Robot. Res.* **40**, 1352–1369 (2021).
56. Pham, H. & Pham, Q.-C. A new approach to time-optimal path parameterization based on reachability analysis. *IEEE Trans. Robot.* **34**, 645–659 (2018).
57. Song, Y., Romero, A., Müller, M., Koltun, V. & Scaramuzza, D. Reaching the limit in autonomous racing: optimal control versus reinforcement learning. *Sci. Robot.* (in the press).

比较实验设置(2)中的性能，该设置具有理想化动力学模型但感知存在噪声。所有基线方法都被提供了与我们用于表征观测噪声的同一高斯过程模型的预测。结果总结在扩展数据表1b中。所有基线方法都受益于更真实的观测，从而获得更高的完成率。然而，我们的方法是唯一能够可靠完成整个赛道的方法。除了观测噪声模型的预测外，我们的方法还考虑了模型的不确定性。关于在受控实验中强化学习与最优控制性能的深入比较，我们建议读者参考文献57。

进行多次迭代的微调

我们研究了不同迭代之间行为变化的程度。我们的分析结果表明，后续的微调操作对性能提升和行为改变的影响可以忽略不计（扩展数据图2）。

在下文中，我们提供了关于此调查的更多细节。我们首先列举微调步骤以提供必要的符号表示：

1. 在仿真中训练策略-0。2. 在真实世界中部署策略-0。该策略基于来自动作捕捉系统的真值数据运行。3. 识别策略-0在真实世界中观察到的残差。4. 基于识别出的残差，对策略-0进行微调以训练策略-1。5. 在真实世界中部署策略-1。该策略仅基于机载传感器测量运行。6. 识别策略-1在真实世界中观察到的残差。7. 基于识别出的残差，对策略-1进行微调以训练策略-2。

我们在各自残差上进行微调后，比较了策略-1和策略-2在仿真中的性能。结果如扩展数据图2所示。我们观察到，与门中心的距离差异（这是衡量策略安全性的一个指标）为0.09 ± 0.08 m。此外，完成单圈所需时间的差异为0.02 ± 0.02 s。请注意，这一单圈时间差异远小于Swift与人类飞行员单圈完成时间之间的差异（0.16 s）。

无人机硬件配置

人类飞行员和Swift所使用的四旋翼飞行器具有相同的重量、形状和推进系统。该平台设计基于Agilicious框架[58]。每个飞行器的重量为870 g，可产生约35 N的最大静态推力，从而得到4.1的静态推重比。每个平台的底座由一个Armattan Chameleon 6英寸主机架构成，该主机架配备T-Motor Velox 2306电机和5英寸三叶螺旋桨。自主无人机的主要计算资源由NVIDIA Jetson TX2及Connect Tech Quasar载板提供，其具有一个运行频率为2 GHz的六核CPU和一个运行频率为1.3 GHz、拥有256个CUDA核心的专用GPU。尽管门检测网络的前向传播在GPU上执行，竞速策略在CPU上评估，一次推理过程耗时8 ms。自主无人机搭载一个Intel RealSense追踪相机T265，其以100 Hz提供VIO估计[59]，并通过USB馈送至NVIDIA Jetson TX2。人类驾驶的无人机既不搭载Jetson计算机也不搭载RealSense相机，而是配备相应的配重。由人类飞行员或Swift生成的形式为集体推力和机体角速率的控制指令被发送至商用飞行控制器，该控制器运行在216 MHz的STM32处理器上。飞行控制器运行Betaflight，这是一款开源飞行控制软件[45]。

人类飞行员的印象

以下引言传达了三位与Swift竞速的人类冠军的印象。

Alex Vanover:

- 这些比赛将在半滚倒转处决出胜负，这是赛道中最具挑战性的部分。
- 这是最棒的一场比赛！我离自主无人机非常近，在试图跟上它时，我真的能感受到湍流。

Thomas Bitmatta:

- 可能性是无限的，这是可能改变整个世界的开端。另一方面，我是一名竞速选手，我不希望有任何东西比我更快。
- 当你飞得越快，你就得用精确度来换取速度。• 看到无人机真正具备的潜力令人鼓舞。很快，AI无人机甚至可以用作训练工具，来理解哪些事情将成为可能。

Marvin Schaepper:

- 与机器竞速的感觉不同，因为你知道机器不会疲倦。

研究伦理

本研究按照《赫尔辛基宣言》开展。根据苏黎世大学的规章制度，本研究方案免于伦理委员会审查，因为未收集任何健康相关数据。参与者在参与研究前均给出了书面知情同意。

数据可用性

评估论文结论所需的所有（其他）数据均包含在论文或扩展数据中。比赛事件的运动捕捉记录及配套分析代码可在 Zenodo 上的文件'racing_data.zip'中找到，网址为 https://doi.org/10.5281/zenodo.7955278。

代码可用性

Swift 的伪代码，详细说明了训练过程和算法，可在 Zenodo 上的文件'pseudocode.zip'中找到，网址为 https://doi.org/10.5281/zenodo.7955278。为防止潜在滥用，本研究相关的完整源代码将不会公开提供。

43. Bauersfeld, L., Kaufmann, E., Foehn, P., Sun, S. & Scaramuzza, D. in *Proc. Robotics:Science and Systems XVII* 42 (Robotics: Science and Systems Foundation, 2021).44. Kaufmann, E., Bauersfeld, L. & Scaramuzza, D. in *Proc. 2022 International Conference onRobotics and Automation (ICRA)* 10504–10510 (IEEE, 2022).45. The Betaflight Open Source Flight Controller Firmware Project. Betaflight. https://github.com/betaflight/betaflight (2022).46. Bauersfeld, L. & Scaramuzza, D. Range, endurance, and optimal speed estimates formulticopters. *IEEE Robot. Autom. Lett.* **7**, 2953–2960 (2022).47. Zhou, Y., Barnes, C., Lu, J., Yang, J. & Li, H. in *Proc. IEEE/CVF* 计算机视觉与模式识别会议（*CVPR*）5745–5753 (IEEE, 2019).48. Pinto, L., Andrychowicz, M., Welinder, P., Zaremba, W. & Abbeel, P. in *Proc. Robotics:Science and Systems XIV* (MIT Press Journals, 2018).49. Molchanov, A. et al. in *Proc. 2019 IEEE/RSJ International Conference on Intelligent Robotsand Systems (IROS)* 59–66 (IEEE, 2019).50. Andrychowicz, O. M. et al. Learning dexterous in-hand manipulation. *Int. J. Robot. Res.***39**, 3–20 (2020).51. Guadarrama, S. et al. TF-Agents: a library for reinforcement learning in TensorFlow.https://github.com/tensorflow/agents (2018).52. Torrente, G., Kaufmann, E., Foehn, P. & Scaramuzza, D. Data-driven MPC for quadrotors.*IEEE Robot. Autom. Lett.* **6**, 3769–3776 (2021).53. Ronneberger, O., Fischer, P. & Brox, T. in *Proc.* 国际医学图像计算与计算机辅助介入会议 234–241 (Springer, 2015).54. Intel RealSense T265 series product family.
https://www.intelrealsense.com/wp-content/uploads/2019/09/Intel_RealSense_Tracking_Camera_Datasheet_Rev004_release.pdf (2019).55. Ryou, G., Tal, E. & Karaman, S. Multi-fidelity black-box optimization for time-optimalquadrotor maneuvers. *Int. J. Robot. Res.* **40**, 1352–1369 (2021).56. Pham, H. & Pham, Q.-C. A new approach to time-optimal path parameterization based onreachability analysis. *IEEE Trans. Robot.* **34**, 645–659 (2018).57. Song, Y., Romero, A., Müller, M., Koltun, V. & Scaramuzza, D. Reaching the limit inautonomous racing: optimal control versus reinforcement learning. *Sci. Robot.* (in thepress).

58. Foehn, P. et al. Agilicious: open-source and open-hardware agile quadrotor for vision-based flight. *Sci. Robot.* **7**, eabl6259 (2022).
59. Jones, E. S. & Soatto, S. Visual-inertial navigation, mapping and localization: a scalable real-time causal approach. *Int. J. Robot. Res.* **30**, 407–430 (2011).

Acknowledgements The authors thank A. Vanover, T. Bitmatta and M. Schaepper for accepting to race against Swift. The authors also thank C. Pfeiffer, T. Längle and A. Barden for their contributions to the organization of the race events and the drone hardware design. This work was supported by Intel's Embodied AI Lab, the Swiss National Science Foundation (SNSF) through the National Centre of Competence in Research (NCCR) Robotics and the European Research Council (ERC) under grant agreement 864042 (AGILEFLIGHT).

Author contributions E.K. formulated the main ideas, implemented the system, performed the experiments and data analysis and wrote the paper. L.B. contributed to the main ideas, the experiments, data analysis, paper writing and designed the graphical illustrations. A.L. formulated the main ideas and contributed to the experimental design, data analysis and paper writing. M.M. contributed to the experimental design, data analysis and paper writing. V.K. contributed to the main ideas, the experimental design, the analysis of experiments and paper writing. D.S. contributed to the main ideas, experimental design, analysis of experiments, paper writing and provided funding.

Competing interests The authors declare no competing interests.

Additional information
Supplementary information The online version contains supplementary material available at https://doi.org/10.1038/s41586-023-06419-4.
Correspondence and requests for materials should be addressed to Elia Kaufmann.
Peer review information *Nature* thanks Sunggoo Jung, Karime Pereida and the other, anonymous, reviewer(s) for their contribution to the peer review of this work. Peer reviewer reports are available.
Reprints and permissions information is available at http://www.nature.com/reprints.

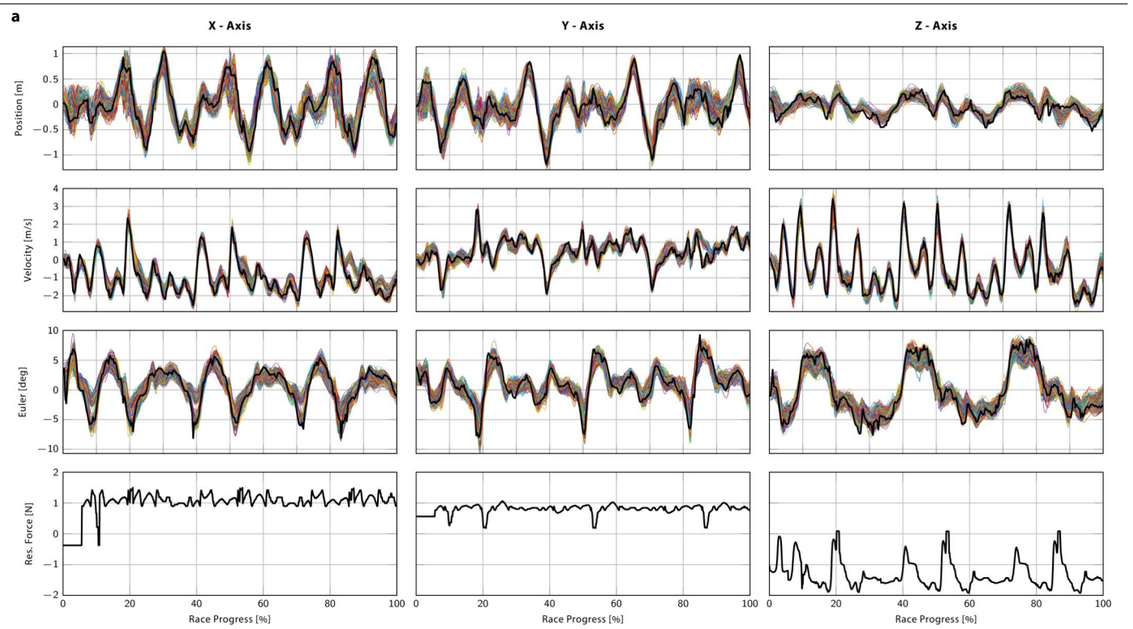
58. Foehn, P. et al. Agilicious：开源开放硬件敏捷四旋翼用于基于视觉的飞行。 *Sci. Robot.* **7**， eabl6259 (2022)。 59. Jones, E. S. & Soatto, S. 视觉惯性导航、建图与定位：一种可扩展的实时因果方法。 *International Journal of Robotics Research* **30**， 407–430 (2011)。

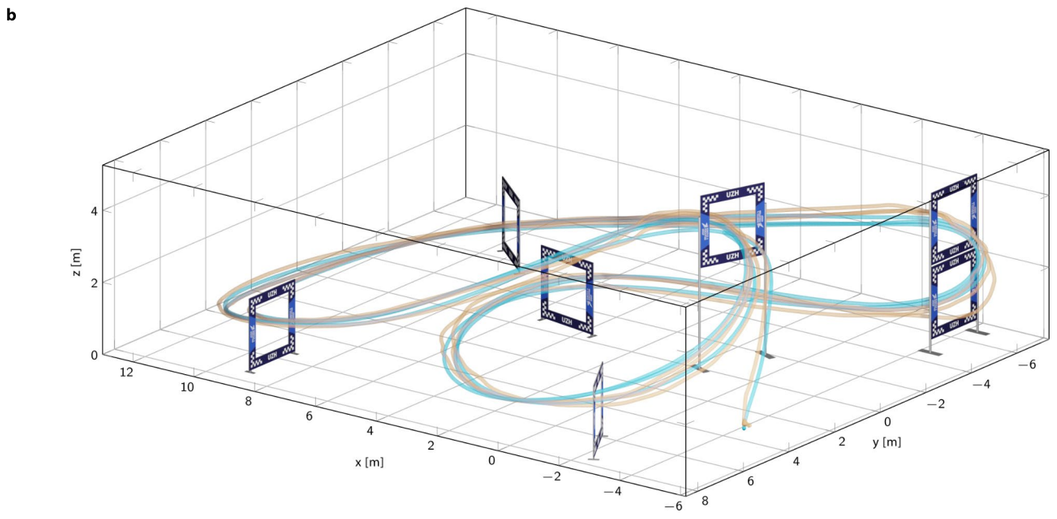
致谢 作者感谢 A. Vanover、T. Bitmatta 和 M. Schaepper 接受与 Swift 竞速。作者还感谢 C. Pfeiffer、T. Längle 和 A. Barden 对赛事组织及无人机硬件设计的贡献。本工作得到英特尔具身智能实验室、瑞士国家科学基金会（SNSF）通过国家研究能力中心（NCCR）机器人学以及欧洲研究委员会（ERC）在资助协议 864042 (AGILEFLIGHT) 下的支持。

作者贡献 E.K.提出了核心思想，实现了系统，完成了实验和数据分析，并撰写了论文。L.B.对核心思想、实验、数据分析、论文写作做出了贡献，并设计了图形插图。A.L. 提出了核心思想，并对实验设计、数据分析和论文写作做出了贡献。M.M.对实验设计、数据分析和论文写作做出了贡献。V.K.对核心思想、实验设计、实验分析和论文写作做出了贡献。D.S.对核心思想、实验设计、实验分析和论文写作做出了贡献，并提供了资助。

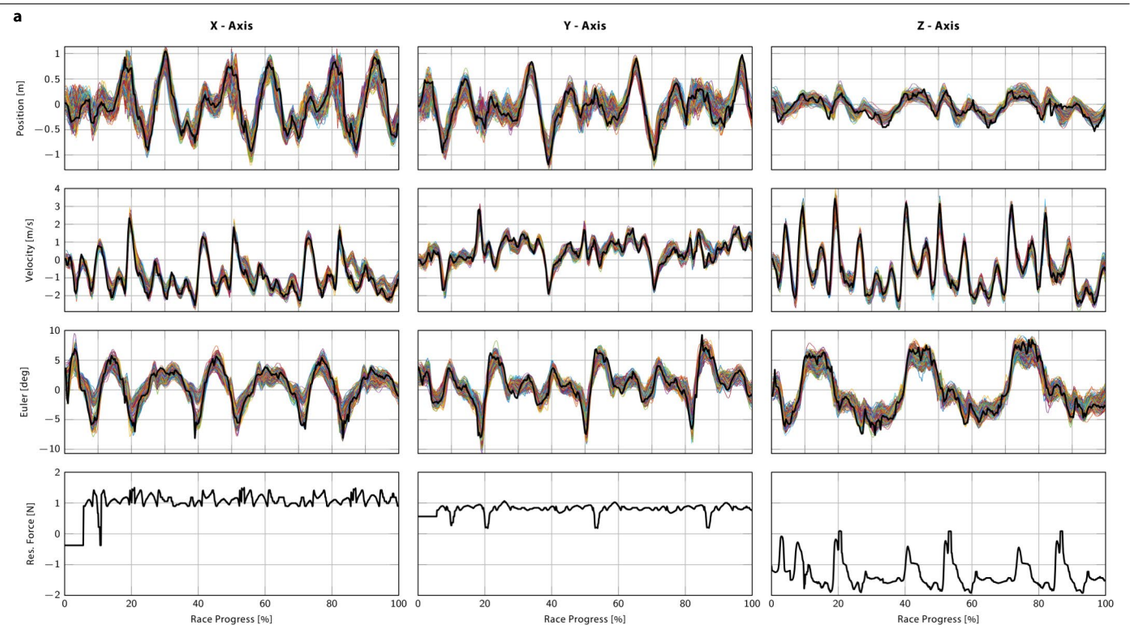
利益冲突 作者声明无利益冲突。

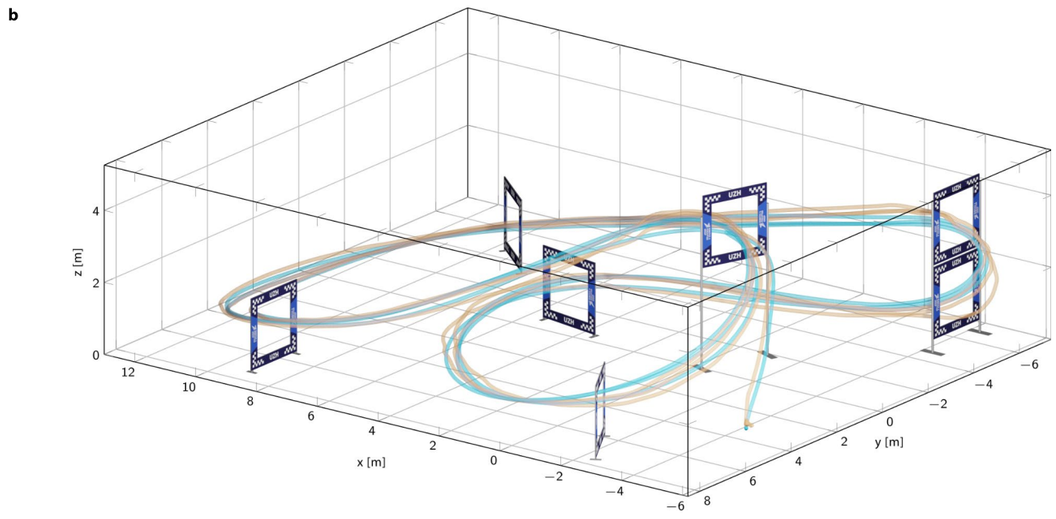
附加信息
补充信息 在线版本包含补充材料，可访问 t
https://doi.org/10.1038/s41586-023-06419-4。**通讯作者及材料索取**请联系 Elia Kaufmann。**同行评审信息** *Nature* 感谢 Sunggoo Jung、Karime Pereida 以及其他匿名审稿人对本工作同行评审的贡献。同行评审报告可获取。**转载与许可信息** 可访问 http://www.nature.com/reprints。



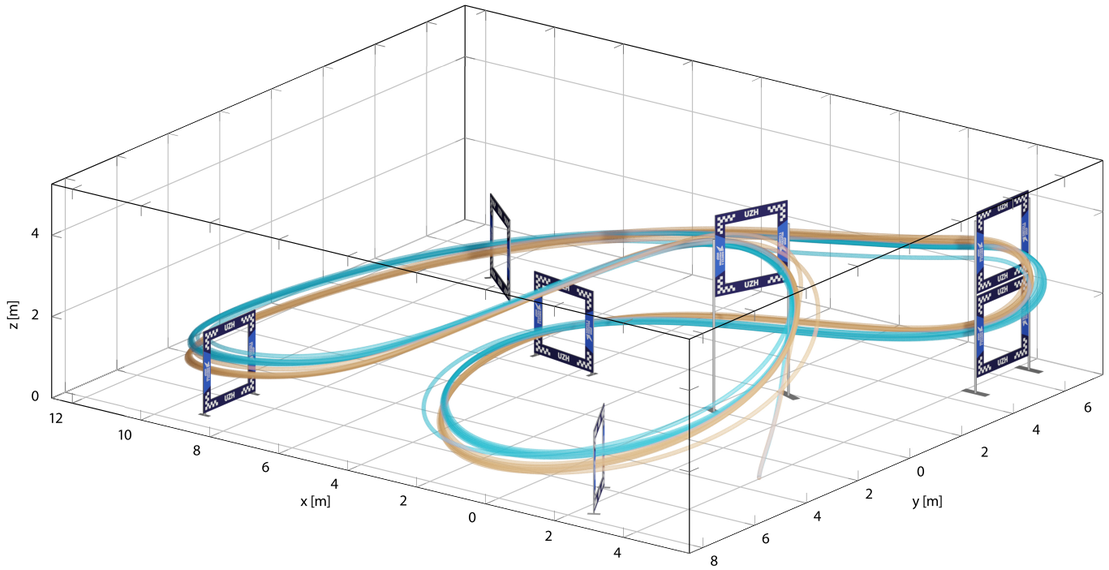


Extended Data Fig. 1 | Residual models. a, Visualization of the residual observation model and the residual dynamics model identified from real-world data. Black curves depict the residual observed in the real world and coloured lines show 100 sampled realizations of the residual observation model. Each plot depicts an entire race, that is, three laps. **b**, Predicted residual observation for a simulated rollout. Blue, ground-truth position provided by the simulator; orange, perturbed position generated by the Gaussian process residual.

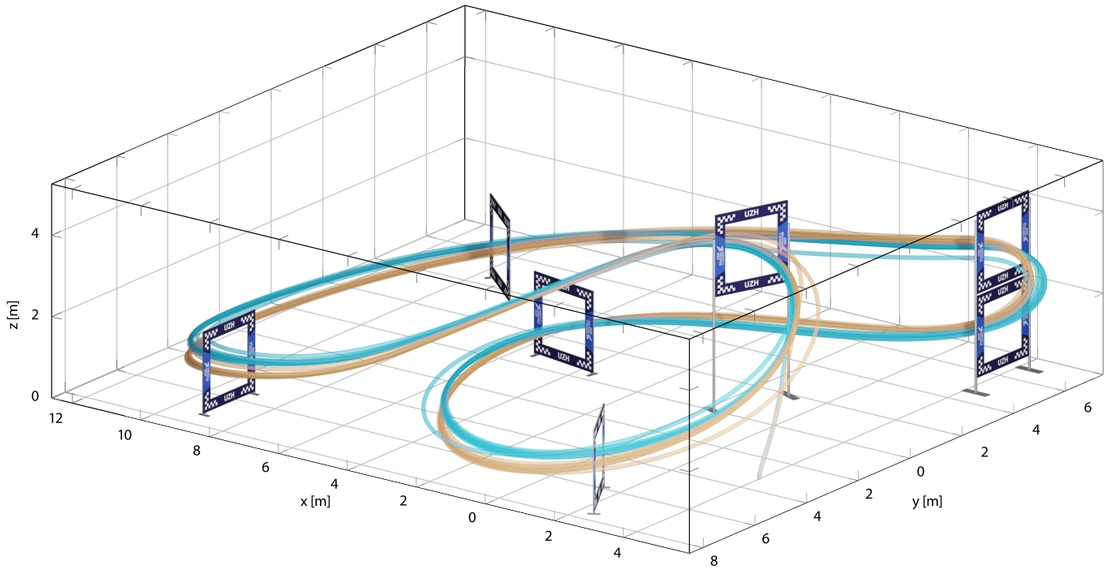




扩展数据图 1 | 残差模型。 a，从真实世界数据中辨识出的残差观测模型和残差动力学模型的可视化。黑色曲线描绘了真实世界中观测到的残差，彩色线条展示了残差观测模型的100个采样实现。每个子图描绘了整场比赛，即三圈。**b**，模拟展开的预测残差观测。蓝色，模拟器提供的真实位置；橙色，高斯过程残差生成的扰动位置。



Extended Data Fig. 2 | Multi-iteration fine-tuning. Rollout comparison after fine-tuning the policy for one iteration (blue) and two iterations (orange).



扩展数据图 2 | 多迭代微调。将策略微调一个迭代（蓝色）和两个迭代（橙色）后的展开比较e).

Extended Data Table 1 | Network parameters and detailed metrics show how our Swift system compares with other approaches

a

Hyperparameter	Value
γ (discount factor)	0.99
ε (importance ratio clipping)	0.2
λ_1	1.0
λ_2	0.02
λ_3	-10.0
λ_4	-2e-4
λ_5	-1e-4

b

Approach	Lap time in seconds	Gate margin in meters	Completion in %
Zero-Shot Transfer [33]	4.92	0.57 — 0.41	42
Domain Randomization	1	0.34 — 0.23	19
Time-Opt. Traj. + MPC [26]	1	0.51 — 0.41	19
Ours	5.26	0.56 — 0.44	100

c

		Ground-truth observations			Noisy observations		
	Approach	Lap time in seconds	Gate margin in meters	Completion in %	Lap time in seconds	Gate margin in meters	Completion in %
Idealized dyn.	Zero-Shot Transfer [33]	4.88	0.63 — 0.46	100	1	n/a — n/a	0
	Domain Randomization	5.06	0.60 — 0.47	100	1	0.43 — 0.30	9
	Time-Opt. Traj. + MPC [26]	4.60	0.50 — 0.25	100	1	0.48 — 0.29	9
	Ours	4.88	0.63 — 0.46	100	5.26	0.56 — 0.44	100
Realistic dyn.	Zero-Shot Transfer [33]	1	0.62 — 0.62	4	1	0.41 — 0.21	9
	Domain Randomization	1	0.28 — 0.28	4	1	0.47 — 0.45	9
	Time-Opt. Traj. + MPC [26]	1	n/a — n/a	0	1	0.23 — 0.23	4
	Ours	5.20	0.51 — 0.30	100	5.42	0.48 — 0.23	100

d

	Autonomous Drone					**Vanover**					**Bitmatta**					**Schaepper**				
	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m
Segment 1	**6.52**	**548.14**	**19.80**	**1.27**	8.27	5.62	348.91	14.96	1.42	7.98	5.93	365.78	15.87	1.36	8.07	4.92	201.06	10.47	1.59	**7.82**
Segment 2	13.72	852.20	29.01	**1.82**	**25.01**	**13.83**	**863.85**	**29.22**	1.83	25.31	13.72	854.36	28.95	1.89	25.94	10.76	508.29	19.97	2.36	25.35
Segment 3	13.70	798.69	27.60	**1.96**	**26.89**	**13.86**	**853.69**	**28.98**	2.09	28.91	13.16	727.36	25.50	2.23	29.35	12.66	636.78	23.47	2.40	30.34
Segment 4	13.43	**1047.05**	**33.69**	1.61	21.62	12.97	953.47	31.62	**1.60**	**20.72**	**13.47**	1035.91	33.39	1.67	22.53	13.11	885.53	29.95	1.77	23.20
Full Race	**13.11**	**866.48**	**29.16**	**17.46**	**228.85**	12.96	843.50	28.65	17.96	232.79	12.89	822.50	27.96	18.74	241.52	11.55	623.53	22.95	21.16	244.49

a, Training hyperparameters. **b**, Comparison with baselines that are provided with the same observation noise model used by our approach. **c**, Evaluation in simulation, with idealized dynamics (top) versus realistic dynamics (bottom) and ground-truth observations (left) versus noisy observations (right). We report the fastest achieved collision-free lap time in seconds, the average and smallest gate margin of successfully passed gates and the percentage of track completed. We compare our approach with a learning-based approach that performs zero-shot transfer, with and without domain randomization during training, as well as a traditional planning and control approach[28]. **d**, Comparison of the average speed, power, thrust, time and distance travelled for each pilot during the fastest flown race. Best numbers are indicated in bold.

扩展数据表1 | 网络参数和详细指标展示了我们的Swift系统与其他方法的比较

a

Hyperparameter	Value
γ (discount factor)	0.99
ε (importance ratio clipping)	0.2
λ_1	1.0
λ_2	0.02
λ_3	-10.0
λ_4	-2e-4
λ_5	-1e-4

b

Approach	Lap time in seconds	Gate margin in meters	Completion in %
Zero-Shot Transfer [33]	4.92	0.57 — 0.41	42
Domain Randomization	1	0.34 — 0.23	19
Time-Opt. Traj. + MPC [26]	1	0.51 — 0.41	19
Ours	5.26	0.56 — 0.44	100

c

		Ground-truth observations			Noisy observations		
	Approach	Lap time in seconds	Gate margin in meters	Completion in %	Lap time in seconds	Gate margin in meters	Completion in %
Idealized dyn.	Zero-Shot Transfer [33]	4.88	0.63 — 0.46	100	1	n/a — n/a	0
	Domain Randomization	5.06	0.60 — 0.47	100	1	0.43 — 0.30	9
	Time-Opt. Traj. + MPC [26]	4.60	0.50 — 0.25	100	1	0.48 — 0.29	9
	Ours	4.88	0.63 — 0.46	100	5.26	0.56 — 0.44	100
Realistic dyn.	Zero-Shot Transfer [33]	1	0.62 — 0.62	4	1	0.41 — 0.21	9
	Domain Randomization	1	0.28 — 0.28	4	1	0.47 — 0.45	9
	Time-Opt. Traj. + MPC [26]	1	n/a — n/a	0	1	0.23 — 0.23	4
	Ours	5.20	0.51 — 0.30	100	5.42	0.48 — 0.23	100

d

	Autonomous Drone					**Vanover**					**Bitmatta**					**Schaepper**				
	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m
Segment 1	**6.52**	**548.14**	**19.80**	**1.27**	8.27	5.62	348.91	14.96	1.42	7.98	5.93	365.78	15.87	1.36	8.07	4.92	201.06	10.47	1.59	**7.82**
Segment 2	13.72	852.20	29.01	**1.82**	**25.01**	**13.83**	**863.85**	**29.22**	1.83	25.31	13.72	854.36	28.95	1.89	25.94	10.76	508.29	19.97	2.36	25.35
Segment 3	13.70	798.69	27.60	**1.96**	**26.89**	**13.86**	**853.69**	**28.98**	2.09	28.91	13.16	727.36	25.50	2.23	29.35	12.66	636.78	23.47	2.40	30.34
Segment 4	13.43	**1047.05**	**33.69**	1.61	21.62	12.97	953.47	31.62	**1.60**	**20.72**	**13.47**	1035.91	33.39	1.67	22.53	13.11	885.53	29.95	1.77	23.20
Full Race	**13.11**	**866.48**	**29.16**	**17.46**	**228.85**	12.96	843.50	28.65	17.96	232.79	12.89	822.50	27.96	18.74	241.52	11.55	623.53	22.95	21.16	244.49

a，训练超参数。**b**，与基线的比较，这些基线使用与我们方法相同的观测噪声模型。**c**，仿真评估，理想化动力学（上）与真实动力学（下）以及真值观测（左）与噪声观测（右）的对比。我们报告最快实现的无碰撞单圈时间（秒）、成功通过门的平均和最小门裕度以及赛道完成度百分比。我们将我们的方法与一种执行零样本迁移的基于学习的方法进行比较，该迁移在训练期间有和没有域随机化，以及一种传统的规划与控制方法[28]。**d**，在最快飞行比赛中每位飞行员的平均速度、功率、推力、时间和飞行距离的比较。最佳数值以粗体表示。